# INTERNSHIP REPORT

**On**

# "COVID-19 ANALYSIS USING POWER BI"

**Internship Organization**: Think NEXT Technologies - industrial training internship Chandigarh Mohali

**SUBMITTED BY:** RINKLE

2024096530 M.Sc.

STATISTICS


**DEPARTMENT OF STATISTICS AND OPERATIONAL RESEARCH,**

**KURUKSHETRA UNIVERSITY, KURUKSHETRA**

**June,2025 – July,2025**

Submitted on :- 6th August 2025

# CERTIFICATE

**This is to certify that Ms.** RINKLE **has partially completed the Semester Training during the period from** 23rd June 2025 **to** 5th August 2025 **in our Organization as a Partial Fulfilment of Degree of Master of Science in Statistics and Operational Research.**

**(Signature of Project Supervisor)**

**Date: _____**

**Ref: TNT/C-24/5472** **Date:--** 20th June 2025

# Internship Letter

Dear Rinkle,

I am writing this letter in response to your application for the Internship in our company. I am Pleased to inform you that you have been selected to join as an intern/Trainee in Data Analyst.

You're joining date will be 22nd June 2025, you are required to work for minimum for 6 weeks, after which your performance will be evaluated by your supervisor.

For any queries, you can contact the HR office during regular working hours. We will be glad to assist you and clarify all your concern.

We wish you a bright future ahead.

Name- Balwinder Kumar
Designation-Admin Executive

For ThinkNEXT Technologies Pvt. Ltd.
Authorised Signatory

Authorized Signatory


ThinkNEXT Technologies Private Limited
ThinkNEXT®
Innovation at every step...
ISO 9001:2015 Certified Company
Indian Iconic AWARD WINNER 2024
Business Excellence AWARD WINNER 2023
Iconic Business Summit AWARDS WINNER 2021
Nation's Business Pride AWARDS WINNER 2021
National Gratitude AWARDS WINNER 2020
Asia's Quality & Entrepreneurship AWARDS WINNER 2019
Business Leaders AWARDS WINNER 2019
NATIONAL ICON AWARDS WINNER 2018
Scan and verify your Certificate
Certificate ID:251486
Ref.No. TNT/C-25/17231
Certificate
Certificate of Excellence
This Certificate do hereby recognizes that
Rinkle D/o Rajesh Kumar
has successfully completed Industrial Training Program
from 23rd June 2025 to 05th Aug 2025
in Data Analyst Grade A
ISO Certified
Member of Confederation of Indian Industry
ISTQB International Software Testing Qualifications Board
Affiliated
For ThinkNEXT Technologies Pvt. Ltd.
Authorised Signatory
Member Training Division
For ThinkNEXT Technologies Pvt. Ltd.
Director
Director
Corporate Office: S.C.F 113, Sector 65, Mohali
A Outstanding 100-90%
B Excellent 89-80%
C Very Good 79-70%
D Good 69-60%
E Satisfactory 59-50%

# DECLARATION

I hereby declare that the Project Report entitled (**"Covid-19 Analysis using Power BI"**) is an authentic record of my own work as requirements of 2nd Sem academic during the period from 2**3rd Jun 2025** to **5th Aug 2025** for the award of degree of Master of Science in Statistics and Operational Research.

**(Signature of student)**
**(RINKLE)**
**(2024096530)**

**Date: _____**

Certified that the above statement made by the student is correct to the best of our knowledge and belief.

## Signatures

## Examined by:

1. 2. 3. 4.

**Head of Department**
**(Signature and Seal)**

# ACKNOWLEDGMENT

I would like to express my heartfelt gratitude to all those who supported me throughout the courses of this project tittle "**Analysis of COVID -19."**

First and foremost, I am extremely thankful to Ms Manpreet Kaur for their valuable guidance ,constant encouragement, and constructive feedback ,which helped me complete this project successfully.

I am also grateful to my peers and family for their motivation and support during the course of this work. This internship has been a great learning experience and has enriched my knowledge ,skills and confidence.

# ABOUT COMPANY

As a second semester student pursuing a degree in Statistics and Operational Research, I had the opportunity to gain valuable industry experience through a six weeks Semester Training program with **Think NEXT Technologies Pvt. Ltd**. During my training, I received a certification in Data Analysis, which provided me with a strong foundation in programming, software development, and statistical analysis. However, my major project revolved around the Covid-19 Analysis using Python and Power BI. This project required extensive work on my part.
Our focus was on creating an analysis report using Power BI to make it easier to understand the visualized form of data. The project demanded independent effort outside of the Institute to ensure its successful completion.

Think NEXT Technologies Private Limited, Mohali is an ISO 9001:2015 certified company. Think NEXT was founded in 2011 and is approved from Ministry of Corporate Affairs and registered under Companies Act 1956. Think NEXT deals in Digital Marketing, Web designing, Web development, Campus ERP Software for Universities/Colleges, eLearning Platform, Chatbots, Mobile Apps Development, Security Systems, Industrial Training, Internships, Online Courses etc. Think NEXT Technologies provides IT solutions using latest technologies e.g. Smart Card (Contact Type, Contactless), NFC, Biometrics, Barcode, RFID, SMS. Think NEXT has its numerous clients across the globe. Over the years, with its hard work, dedication, and commitment, Think NEXT has been emerged as one of the leading IT companies in Chandigarh region. Think NEXT is Google Partner for Google AdWords, Bing Ads Accredited, HubSpot Certified, Facebook Blueprint Certified, Microsoft Bing Ads Accredited Company. Over the years, with its hard work, dedication, and commitment, Think NEXT is becoming famous in the field of Digital Marketing, Web Development, and Industrial Training. Founder and Director, Er. Munish Mittal, MTech (IT), BTech (CSE), himself is having vast experience of more than 20 years in software development and digital marketing. He is also the author of two books.

# ABSTRACT

In 2020 the world has generating 52 times the amount of data as in 2010, and 76 times the number of information sources. Being able to use this data provides huge opportunities and to turn these opportunities into reality, people need to use data to solve problems. Unfortunately, during a global pandemic, when people throughout the world are looking for reliable, trustworthy information about COVID-19(Coronavirus). In this scenario Power BI play the vital role, because Power BI is an extremely powerful tool for visualizing massive sets of data very easily. It has an easy to use drag and drop interface. You can build beautiful visualizations easily and in a short amount of time. Power BI supports a wide array of data sources. COVID-19(Coronavirus) analytics with Power BI, we will create dashboards that helps in identify the story within our data, and we will better understand the impact of COVID-19 (Coronavirus).

The Power BI are the tools which deals with the big data analytics also it generates the output in visualization technique with data sources and DAX query language through dashboard in Power BI,
i.e., more understandable, and presentable. Its features include data blending, real-time reporting, and collaboration of data. The project aims to show on how we can use the Power BI with analytics data using DAX query language and its performance on presenting the dashboard. So, in this project, I have created an analytical dashboard to know the trend and performance, and to know which country is affected mostly, which are the top regions and which country having more active cases.

This project represents the large dataset into visualization form to quickly see the insights of all the countries. In the end, this report gives the clear picture of growing COVID-19(Coronavirus) data and the tools which can help more effectively, accurately, and efficiently.

# TABLE OF CONTENTS

# LIST OF TABLES

# LIST OF FIGUES

**Figures**
**Page No.**

# CHAPTER 1 Introduction

## 1.1. Brief Overview of Work

The COVID-19 pandemic has underscored the global nature of health crises, affecting individuals in nearly every part of the world. Power BI's utility in analysing and visualizing COVID-19 data can be gauged by its performance, user-friendly environment, and speed. Designed to facilitate the creation of visuals and graphics without the need for programming knowledge, Power BI democratizes data visualization. It serves as an intuitive medium for users to easily interpret and analyse data, particularly in the context of big data.

Power BI's ability to connect with a variety of data sources and its support for data blending and realtime collaboration sets it apart as a unique tool. This paper introduces Power BI and outlines the process of using it for the interactive visualization and analysis of COVID-19 data, advocating for its widespread adoption. As a modern data analytics and visualization tool, Power BI offers flexibility, ease-of-use, and a smooth experience for users, enhancing the quality of governance policies and decision-making processes.

## 1.2. Objective

The aim of the project is to create a report to help the Covid Analytics to maintain the records of a large data of cases, handle cases details, and can understand how the cases increased. Covid analytics deals with the maintenance of cases and mortality rate in India as well as globally. This Covid analytics system is user friendly. Having high performance and no time consuming.

The main objective in making this dashboard is that the reader can directly analyse what we had in past years records, which areas are having high cases, or which is having high recoveries, etc.

## 1.3. Scope

The **scope of COVID-19 analysis using Power BI** is quite extensive. Let's explore the key aspects:

1. **Data Collection and Integration**:

- Gather COVID-19 data from reliable sources (such as WHO, GitHub, or government health departments).
- Integrate data related to cases, deaths, recoveries, testing, vaccinations, and other relevant metrics. ○ Ensure data quality by cleaning and transforming it using Power Query.

2. **Data Modelling and Relationships**:

○ Create a robust data model in Power BI Desktop. ○ Define relationships between tables (e.g., cases, demographics, testing) to enable meaningful analysis. ○ Use DAX (Data Analysis Expressions) to create calculated columns and measures.

3. **Visualization and Dashboard Creation**: ○

Design interactive visualizations:

- ✦ Line charts for time-series trends.
- ✦ Maps to show regional variations.
- ✦ Bar/column charts for comparisons.
- ✦ Pie charts for proportions.

○ Include slicers and filters for user interactivity. ○ Build a comprehensive dashboard with multiple pages, each focusing on specific aspects (e.g., cases, testing, vaccinations).

4. **Geospatial Analysis**:

- Use maps to visualize COVID-19 spread across regions. ○
- Highlight hotspots, clusters, and trends. ○ Overlay demographic data (population density, age groups) for deeper insights.

5. **Time-Series Analysis**:

- Explore daily, weekly, or monthly patterns.

- Identify spikes, trends, and seasonality.
- Calculate growth rates and doubling times.

6. **Demographic Insights**:
   - Analyse how COVID-19 affects different age groups, genders, and ethnicities.
   - Compare mortality rates across demographics.
   - Consider socioeconomic factors.

7. **Predictive Modelling**:
   - Use historical data to build predictive models (e.g., time-series forecasting).
   - Forecast future cases, hospitalizations, or vaccination rates.
   - Evaluate model accuracy and adjust as needed.

8. **Report Writing and Storytelling**:
   - Create a narrative around your findings.
   - Explain trends, anomalies and policy implications.
   - Use visuals and text to convey insights effectively.

## 1.4. Project Dashboard

### 1.4.1 Globe

Dashboard describe the covid-19 analysis for the global data using various visualisations and slicers for filtering the data accordingly.

### 1.4.2 India

Dashboard describe the covid-19 analysis for the Indian data using various visualisations and slicers for filtering the data accordingly.

## 1.5. Project Requirements

Developing a Covid-19 Analysis project necessitates a fusion of suitable software tools and hardware resources to efficiently gather, handle, scrutinize, and present the vast datasets implicated. Here is an elaborate breakdown of the software and hardware prerequisites for constructing such a project:

### 1.5.1 Hardware Platform

The system requires the following hardware:

3

- **Processor:** i3 or above
- **Processor speed:** 2.00GHz
- **CPU RAM:** 8GB or above
- **Hard disk:** 1TB or above
- **Internet Connection:** Yes
- **Display:** 1024 * 768, True Type Color-32 Bit
- **Mouse:** Any Normal Mouse.
- **Keyboard:** Any window Supported Keyboard.

### 1.5.2 Software Platform

The system requires the following hardware:

- **IDE:** Jupyter Notebook is an open-source IDE used for this project.
- **Operation System:** Windows or any equivalent can be used for the project.
- **Language:** Python is a versatile programming language with extensive libraries for data manipulation and analysis.
- **Power BI:** Power BI is a business analytics service provided by Microsoft. It aims to provide interactive visualizations and business intelligence capabilities to create their own reports and dashboards for the end users.
- **Libraries:** Different Python libraries listed below are used for cleaning, transforming, structuring data and for creating visualizations during data exploration.

- Pandas
- matplotlib.pyplot
- seaborn

# CHAPTER 2

## System Analysis

AIM and objective of this project is: The aim of the project is to create a report to help the Covid Analytics to maintain the records of a large data of cases, handle cases details, and can understand how the cases increased. Covid analytics deals with the maintenance of cases and mortality rate in India as well as globally. This Covid analytics system is user friendly. Having high performance and no time consuming.

### 2.1 Literature Review

The **COVID-19 pandemic** has reshaped our world, affecting lives, economies, and healthcare systems globally. Amidst this crisis, data science emerges as a powerful tool to understand the virus's impact, predict trends, and inform decision-making.

In this project, we delve into the realm of data analysis using Python. Our goal is to explore COVID-
19 datasets, extract meaningful insights, and visualize patterns. Let's walk through the essential steps:

1. **Importing Required Packages**: ○ Python, with its rich ecosystem of libraries, empowers us. We'll use **Pandas** and **NumPy** for data wrangling, and **Seaborn** and **Matplotlib** for visualization.
   - Importing these packages sets the stage for our analysis.
2. **Gathering Data**:
   - Quality data is the bedrock of analysis. We've collected COVID-19 data from various sources. ○ Our primary dataset includes cumulative confirmed cases per day in each country.
3. **Data Wrangling**:
   - Raw data needs transformation. We'll clean, reshape, and prepare it for analysis.
4. **Exploratory Data Analysis (EDA) and Visualization**:
   - EDA unveils insights. We'll:
     ✦ Examine time series data. ✦ Visualize patterns.

**Importance of Covid-19 Analysis**

The analysis of COVID-19 data holds **critical importance** for several reasons:

1. **Public Health Decision-Making**: ○ **Informed decisions**: Analysing COVID-19 data helps policymakers, healthcare professionals, and governments make informed decisions related to containment measures, resource allocation, and vaccination strategies.
   - **Predictive modelling**: By studying trends, we can predict potential outbreaks, allocate medical supplies, and plan for surge capacity.
2. **Epidemiological Understanding**: ○ **Transmission patterns**: Analysing data reveals how the virus spreads, its incubation period, and the role of asymptomatic carriers. ○ **Risk factors**: Identifying vulnerable populations (e.g., elderly, immunocompromised) helps tailor interventions.

3. **Monitoring and Surveillance**: ○ **Early detection**: Regular analysis allows us to detect outbreaks early, preventing exponential spread.
○ **Tracking variants**: Monitoring variants helps adapt vaccination strategies and treatment protocols.

4. **Resource Optimization**:
○ **Healthcare system planning**: Data analysis guides hospital capacity planning, ventilator allocation, and staffing decisions. ○ **Supply chain management**: Efficiently distributing medical supplies (PPE, ventilators, vaccines) relies on data insights. 5. **Communication and Education**:
○ **Public awareness**: Clear visualizations help educate the public about infection rates, preventive measures, and vaccination progress.

## 2.2 Project Feasibility Study

### 2.2.1. Technical Feasibility

Technical feasibility study is concerned with specifying equipment and software that will successfully satisfy the user requirement; the technical needs of the system may vary considerably. The facility to produce outputs in each time. Our project is an analysis report which is based on data provided. In this, every dashboard as output is render from the data so it is necessary that the dashboard should be rendered in time.

### 2.2.2. Economical Feasibility

Economical feasibility is the measure to determine the cost and benefit of the proposed system. A project is economical feasible which is under the estimated cost for its development. These benefits and costs may be tangible or intangible. Covid-19 Analysis is the cost-effective project in which there is less possibility of intangible cost so there is no difficulty to determine the cost of the project.

### 2.2.3. Operational Feasibility

Operation feasibility is used to check whether the project is operationally feasible or not. Our project is mainly different from the other system because of its Power BI feature. So, the measure for operational feasibility is something different from other system. Generally, the operational feasibility is related to organization aspects.

The change determination is as such that early product were either a man or group of men or the manual analysis but now a day with the advent of Internet technology**.**

## 2.3. Project Timeline

**1. Month**

**1: Project Initiation and Planning**

- Conduct initial project kick-off meeting to define project objectives, scope, and deliverables.
- Develop project plan, including task breakdown, resource allocation, and timeline estimation.
- Complete literature review and research on covid-19, data sources, and relevant methodologies.

**2. Month**

**2: Data Collection and Preprocessing**

- Gather relevant datasets containing patient information, cases records, and country/region results.
- Clean, preprocess, and transform the dataset to ensure data quality and integrity.
- Clean and organize the data. Make sure you have a date column in a format that Power BI can recognize.

**3. Month**

**3: Exploratory Data Analysis (EDA)**

- Perform exploratory data analysis (EDA) to gain insights into the dataset's characteristics and distributions.
- Designate data analysts to build the data model, including creating relationships between different data sets.
- Task individuals with creating DAX measures for calculating key metrics like case fatality rate, recovery rate, and active cases.

**4. Month**

**4: Visualisation, Visuals integration and Dashboard creation**

- Divide the team into groups responsible for creating interactive charts, maps, and tables for the dashboard.
- Adjust the visual properties to display the data clearly. This may include setting the date range, defining milestones, and choosing colours to represent different data points.
- Utilize Power BI's slicers and filters to allow users to interact with the timeline. This can help viewers focus on specific time periods or events.
- Highlight significant dates, such as the start of lockdowns or vaccine rollouts, to provide context to the data.
- Collaborate to put together the different visual components into a cohesive dashboard.

**5. Month**

**5: Documentation, and Presentation**

- Ensure proper documentation of the data sources, methodologies, and any assumptions made during the analysis.
- Check your timeline for accuracy and clarity. Adjust as necessary to ensure it communicates the intended message effectively.
- Review the dashboard for accuracy and completeness before publishing it for use.

## 2.4. Project Timeline Chart

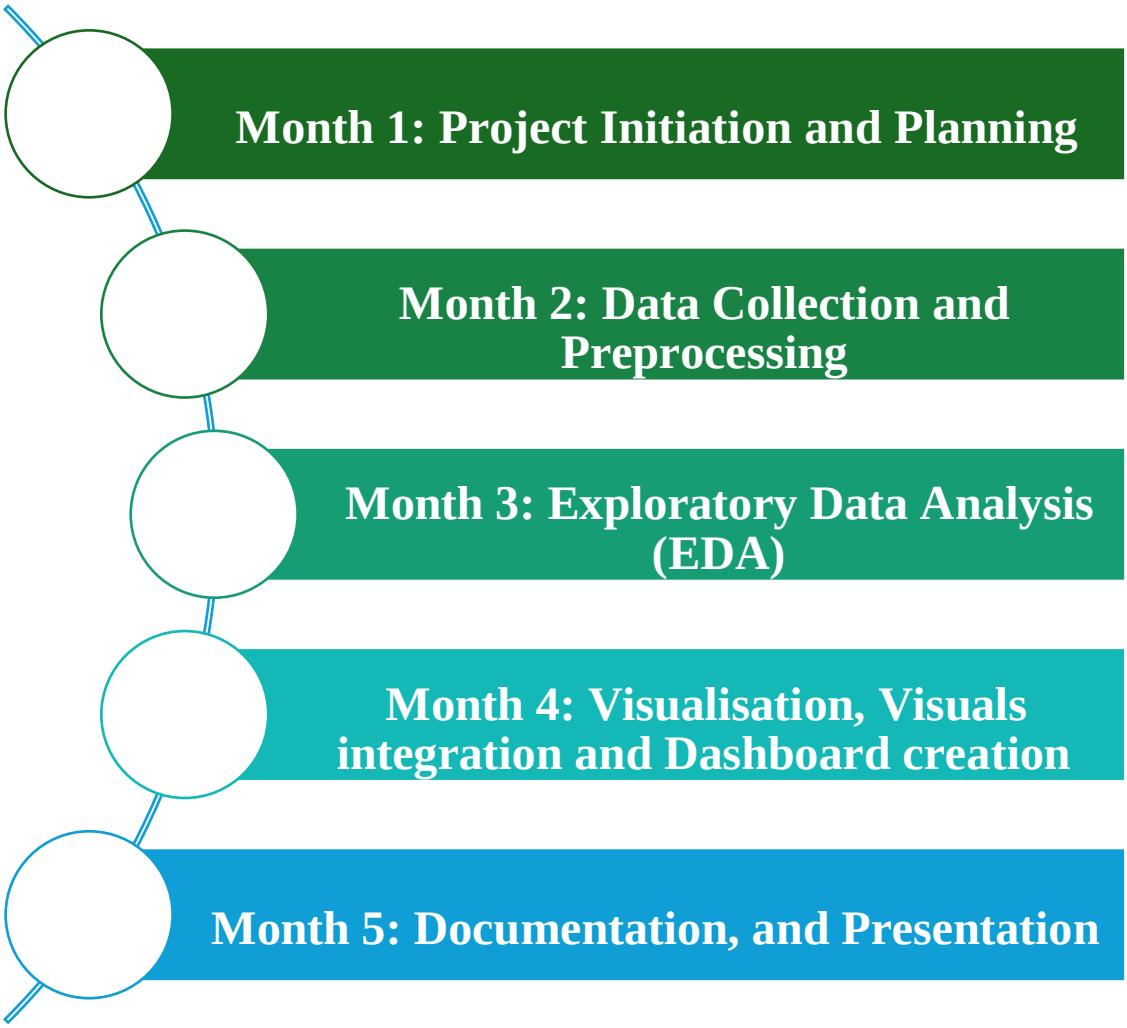



Fig 2.1 Project Timeline Chart

## 2.5. Scalability and performance

The scalability of Power BI High quality/elite capacity, and the scalability of backend is Direct Query or Live Connect data sources. The tool is meant to be used in capacity planning and scale evaluation scenarios when admins of Power BI capacities and backend data sources wish to test the ability of their architecture to serve a certain scale. “It has a 1 GB limit per dataset that is imported into Power BI. If you have chosen to keep the Excel experience, instead of importing the data, the limit is 250 MB for the dataset”.

## 2.6. Accessibility

When Power BI Dashboard is built on a specific topic with accessible dashboards and reports, the accessibility can be given to many persons in the world. For accessibility, you must share your report on the Power BI web service so everyone could see it.

## 2.7. GUI

In **Power BI**, the term **GUI** stands for **Graphical User Interface**.

**Power BI Desktop:**

When you create reports and dashboards using **Power BI Desktop**, you interact with its **graphical user interface (GUI)**. The GUI provides a visual environment where you can design and build your data models, create visualizations, and define relationships between data elements.

Key components of the Power BI Desktop GUI include:

- **Tabs and Ribbon**: These allow you to access various features and functionalities.
- **Data View**: Where you manage your data sources, transform data, and create calculated columns.
- **Report View**: Where you design visualizations and arrange them on the canvas.
- **Fields Pane**: Lists all available fields from your data sources.
- **Visualizations Pane**: Allows you to choose and customize visualizations.
- **Filters and Slicers**: Used to filter data dynamically.
- **Formula Bar**: Where you write DAX (Data Analysis Expressions) formulas.
- **Canvas**: The workspace where you create your report layout.

# CHAPTER 3

# Software Tools

## 3.1. Overview Of Python Technology

### 3.1.1. History of Python

The Python programming language was conceived in the late 1980s by Guido van Rossum at CWI in the Netherlands. Its implementation began in December 1989 as a successor to ABC, with features for exception handling and interfacing with the Amoeba operating system. The name "**Python**" was inspired by the British TV show Monty Python's Flying Circus. Here are some key milestones in Python's history:

**1. Early History:**

- In February 1991, Van Rossum published the code (version 0.9.0) to alt.sources. This initial release already included classes, inheritance, exception handling, functions, and core data types like lists and dictionaries.
- Python's exception model resembled that of Modula-3, with the addition of an else clause.
- The comp.lang.python discussion forum was formed in 1994, marking a milestone in Python's userbase growth.

**2. Version 1:**
- Python reached version 1.0 in January 1994. Major new features including frictional programming tools like lambda, map, filter, and reduce.

**3. Python 2.0:**

- Released on October 16, 2000, Python 2.0 introduced significant features:
  - Cycle-detecting garbage collector (in addition to reference counting) for memory management.
  - Unicode support.
  - A shift to a more transparent and community-backed development process.

**4. Python 3.0:**

- A major, backwards-incompatible release.
- Released on December 3, 2008, after extensive testing.
- Many features were backported to the now-unsupported Python 2.6 and 2.7

Python's evolution continues, and it remains a popular and versatile language in the world of programming.

### 3.1.2. Features of Python

Python is a powerful and versatile programming language with several key features:

1. **Free and Open Source**: Python is freely available, and its source code can be accessed by the public. You can download it from the official website.
2. **Easy to Code and Read**: Python's high-level syntax makes it easy to learn and write code. You don't need to remember system architecture or manage memory. Code blocks are defined by **indentations**, not semicolons or brackets.
3. **Object-Oriented Language**: Python supports object-oriented programming, including classes, object encapsulation, and inheritance.
4. **GUI Programming Support**: You can create graphical user interfaces using modules like **PyQt5**, **wxPython**, or **Tkinter**.
5. **Large Community Support**: Python has a vast community on platforms like **Stack Overflow**, where questions are answered promptly.
6. **Easy to Debug**: Python provides excellent error tracing information, making it easier to identify and correct issues.
7. **Portability**: Python is a portable language; code written for one platform (e.g., Windows) can run on others (Linux, Unix, Mac) without modification.
8. **Integrated Language**: Python can be easily integrated with other languages like C, C++, etc.
9. **Interpreted Language**: Python interprets code line by line, allowing for quick development and debugging.
10. **Versatile and Extensible**: Python's extensive library of open-source packages allows you to build complex software efficiently.

### 3.1.3. Python and Visualization

**Python** is a versatile programming language known for its readability and ease of use. When it comes to **data visualization**, Python offers several powerful libraries. Some of the most popular ones are:

1. **Matplotlib**:

- Matplotlib is a widely used 2-D plotting library in the

Python community. ○ It allows you to create various types of visualizations, including line charts,

bar charts, pie charts, histograms, scatterplots, and more.

- You can embed Matplotlib plots in applications using GUI toolkits like Tkinter, etc.

2. **Seaborn**:

- Built on top of Matplotlib, Seaborn provides a high-level interface for creating statistical graphics.
- It simplifies complex visualizations and enhances the aesthetics of plots. ○ Seaborn is particularly useful for exploring relationships between variables and creating attractive color palettes.

3. **Plotly**:

- Plotly is an open-source graphing library that allows you to create interactive web-based visualizations.
- You can display Plotly plots in Jupyter notebooks, web applications using Dash, or save them as individual HTML files.
- It's great for creating dynamic charts, maps, and other interactive elements.

4. **Bokeh**:

- Bokeh is another interactive visualization library that targets modern web browsers.
- It generates interactive plots with smooth zooming,

panning, and tooltips. ○ Bokeh is well-suited for creating interactive dashboards and visualizations for the web.

5. **Pygal**:

○ Pygal is a simple yet powerful library for creating SVG-based interactive charts. ○ It's lightweight and easy to use, making it suitable for small-scale projects or quick visualizations.

### 3.1.4. Python and Analysis

Python supports object-oriented programming, has a large community, and is easy to debug. Additionally, it's an interpreted language, making development efficient.

When it comes to **data analysis**, Python offers several powerful libraries:

1. **NumPy**: A widely used open-source library for scientific computation. It supports multidimensional data, large matrices, and efficient mathematical functions. NumPy arrays are preferred over lists due to memory efficiency and convenience.
2. **Pandas**: Essential for data analysis, manipulation, and cleaning. Features include DataFrames (for efficient data manipulation), tools for reading/writing data in various formats, and integrated indexing.
3. **Matplotlib**: A versatile 2-D plotting library for creating various types of charts.
4. **Seaborn**: Built on Matplotlib, Seaborn simplifies statistical graphics and enhances aesthetics. Ideal for exploring relationships between variables.
5. **Scikit-learn**: A machine learning library for classification, regression, clustering, and more. Widely used for data analysis tasks.
6. **Statsmodels**: Useful for statistical modeling and hypothesis testing. Provides tools for regression, ANOVA, and more.

### 3.1.5. Python Virtual Machine

he Python Virtual Machine (PVM), also known as the Python interpreter, plays a crucial role in executing Python code. Before execution, Python source code is compiled into bytecode. This bytecode serves as a platform-independent intermediate representation of the Python code. The heart of the PVM is the interpreter loop. It fetches bytecode instructions, decodes them, and executes them sequentially. The PVM maintains a Python object model to represent data types (like integers, strings, and lists). It handles object creation, manipulation, and

destruction during execution. The PVM manages memory allocation and garbage collection for Python objects dynamically.

The PVM loads bytecode from compiled .pyc files or directly from memory (if the code is generated dynamically). The interpreter loop fetches bytecode instructions, interprets them, and executes corresponding operations. It maintains a stack to store intermediate values and operands. Bytecode instructions cover loading values, arithmetic computations, function calls, and control flow (e.g., jumps, loops). PVM handles namespaces, function calls, exceptions, and module imports.

### 3.1.6. Paradigm of Python

Python supports four main programming paradigms:

Imperative Paradigm: This paradigm emphasizes step-by-step instructions. It involves specifying how to achieve a task by providing explicit commands. Imperative programming is common in procedural languages.

Object-Oriented Paradigm (OOP): In OOP, objects are the central building blocks. Objects encapsulate data (attributes) and behaviour (methods/functions). Python's OOP features include classes, inheritance, and polymorphism.

Functional Paradigm: Functional programming treats computation as the evaluation of mathematical functions. Python supports functional programming through features like lambda functions, map, filter, and reduce.

Procedural Paradigm: Procedural programming organizes code into functions or procedures. It focuses on modularization and step-by-step execution. Python allows procedural-style coding alongside other paradigms.

## 3.2. Python Libraries

### 3.2.1. NumPy

NumPy, short for Numerical Python, is an open-source Python library that is integral in the field of scientific computing. It provides a high-performance multidimensional array object and tools

for working with these arrays. NumPy arrays are faster and more compact than Python lists, offering an array of operations that can be performed on massive quantities of data efficiently and with ease.

The library is not only the cornerstone of scientific computing in Python but also a foundational package that supports a wide array of operations. These include complex mathematical functions like linear algebra, Fourier transforms, and random number generation, all of which can be applied to arrays for robust data analysis and manipulation.

NumPy's capabilities make it a critical tool for researchers, data scientists, and engineers who require high-level mathematical functions to solve various problems. Its array-processing package is versatile, allowing it to serve as an efficient multi-dimensional container of generic data, which is essential for handling the vast datasets commonly encountered in data-driven fields today.

Moreover, NumPy's compatibility with a range of other libraries, such as Pandas, SciPy, and Matplotlib, ensures that it remains a staple in the data analysis workflow, providing a seamless experience from data preprocessing to visualization3. Its significance is further underscored by its widespread use in academia and industry, solidifying its position as a fundamental tool in the Python programming ecosystem.

### 3.2.2. Pandas

Pandas is a highly esteemed Python library widely utilized for data manipulation and analysis. It provides high-level data structures and a vast array of tools for cleaning, transforming, and analysing data. With its core data structure, the DataFrame, Pandas enables users to manage and manipulate structured data with ease. The library simplifies tasks such as data import from various file formats, missing data handling, data wrangling, and merging datasets.

One of the key strengths of Pandas is its ability to work with different data types and seamlessly manage missing values, which is essential for preparing real-world data for analysis. It also offers powerful time series functionality, making it the go-to tool for financial and economic data analysis. Moreover, Pandas integrates well with other libraries in the Python ecosystem, such as

NumPy for numerical computations and Matplotlib for plotting, creating a robust environment for data science tasks.

Pandas' syntax is intuitive and readable, which makes it accessible to newcomers while still being powerful enough for advanced users. Its comprehensive documentation and active community support further contribute to its popularity among data professionals. Whether it is for academic research, industry applications, or hobbyist projects, Pandas stands out as a foundational tool in the Python data analysis landscape.

### 3.2.3. Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It is a multi-platform data visualization tool that provides the ability to plot a wide range of graphs and charts with just a few lines of code. Matplotlib is particularly useful in scientific computing and data analysis for visualizing complex data patterns and relationships through its robust plotting functions.

The library's versatility allows for the creation of simple bar charts, histograms, scatter plots, and more complex plots like contour plots and 3D graphics. One of Matplotlib's strengths is its ability to customize every aspect of a figure, enabling users to achieve the precise look they desire for their plots. This includes adjusting the colours, line styles, axes, fonts, and layout.

Matplotlib integrates well with other data science libraries such as NumPy and Pandas, making it a principal component of the data visualization process in Python. It is also the foundation upon which libraries like Seaborn and Plotly are built, extending its capabilities further. Whether for exploratory data analysis, scientific research, or developing sophisticated data-driven reports, Matplotlib remains a reliable and powerful tool in the Python ecosystem.

### 3.2.4. Seaborn

Seaborn is a powerful data visualization library in Python that builds on Matplotlib and is tightly integrated with Pandas data structures. It is designed to create attractive and informative statistical graphics with ease. Seaborn simplifies the process of generating complex visualizations like

heatmaps, time series, and violin plots, which are essential for exploring and understanding data patterns and relationships.

The library provides a high-level interface for drawing attractive statistical graphics and is particularly suited for making complex plots more accessible and understandable. Seaborn's default styles and colour palettes are designed to enhance the aesthetic appeal and readability of plots, which is beneficial when presenting data to an audience.

One of the key features of Seaborn is its ability to work seamlessly with DataFrame objects, allowing for direct plotting of data from these structures. This makes it an invaluable tool for data analysts who need to visualize data quickly and effectively. Whether you are a seasoned data scientist or a beginner, Seaborn's intuitive syntax and rich functionalities make it an excellent choice for statistical data visualization in Python.

## 3.3. Jupyter Notebook

Jupyter Notebook is a versatile web-based platform that combines live code, narrative text, visualizations, and more. It allows users to create interactive documents where they can write and execute code in over 40 programming languages, including Python, R, and Julia. These notebooks are widely used for data analysis, scientific research, and machine learning. JupyterLab, the nextgeneration interface, offers flexibility and customization, while the classic Jupyter Notebook provides a streamlined experience. Additionally, Voilà transforms notebooks into standalone web applications for sharing insights and results. It is an interactive computing environment that has become indispensable for data scientists, researchers, and developers. At its core, Jupyter allows users to create documents (called notebooks) that seamlessly blend code, visualizations, and explanatory text. Jupyter supports over 40 programming languages, with Python being the most popular. Users can write and execute code cells within the notebook, making it ideal for data analysis, machine learning, and scientific computing. Unlike traditional code editors, Jupyter notebooks produce rich output. This includes visualizations (such as plots and charts), LaTeX equations, images, and even interactive widgets. It's a powerful tool for communicating complex ideas. Notebooks allow users to intersperse code with narrative text. This means you can explain your thought process, provide context, and document your work—all in one place. It's like having

a lab notebook for your computational experiments. Jupyter notebooks can be easily shared via email, GitHub, or other platforms. Colleagues can reproduce your analyses, tweak parameters, and build upon your work. JupyterHub enables multiuser scenarios, making it suitable for teams. JupyterLab, the next-generation interface, provides a flexible environment where users can arrange notebooks, code consoles, and other tools in a customizable layout. It's a step forward from the classic notebook interface. In summary, Jupyter Notebook empowers users to explore data, prototype algorithms, and communicate findings—all within a single, interactive document.

**Features:**

1. **Live Code Execution**: ○ Jupyter allows you to write and run code interactively. You can execute code cells and see the output immediately. ○ It supports over 40 programming languages, including Python, R, Julia, Ruby, Scala, and more.

2. **Rich Output**:

- Jupyter notebooks produce rich output, including text, images, plots, and LaTeX equations.
- You can visualize data, create charts, and display results in various formats.

3. **Narrative Text**: ○ Markdown cells allow you to add narrative text, headers, lists, and hyperlinks to your notebook.

- Explain your thought process, document your work, and enhance readability.

4. **Interactive Widgets**:

○ Jupyter supports interactive widgets (e.g., sliders, buttons) that allow users to manipulate parameters and visualize changes dynamically. ○ Useful for exploring data or experimenting with models.

5. **Web-Based Interface**:

- Jupyter Notebook runs in your web browser, making it accessible from anywhere.
- You can create, edit, and share notebooks online.

6. **Kernels**: ○ Kernels are independent processes that execute user code in specific languages (e.g., Python, R).

- They return results to the notebook interface, enabling multi-language support.

**Applications**

Jupyter Notebook is an incredibly versatile tool that has a wide range of applications in various fields. Here are some of the key applications:

- **Data Science Workflows:** Jupyter Notebooks are extensively used for organizing and documenting the steps involved in data analysis, from data cleaning to the application of statistical models and machine learning algorithms.
- **Educational Purposes:** They serve as an excellent educational tool for teaching programming and data science, allowing for the combination of explanatory text, code, and output in a single document.
- **Research Documentation:** Researchers use Jupyter Notebooks to create reproducible research papers, as they can include the narrative, the code, and the results all in one place.
- **Presentation and Reporting:** Notebooks can be converted into slideshows, PDFs, and other formats, making them useful for presentations and reporting.
- **Interactive Computing:** With support for over 40 programming languages, including Python, R, Julia, and Scala, Jupyter Notebooks are ideal for interactive computing tasks.
- **Machine Learning Model Development:** They are widely used for developing and evaluating machine learning models, providing a space to experiment with different algorithms and parameters.
- **Collaborative Projects:** Jupyter Notebooks facilitate collaboration among data scientists and analysts, as they can be shared and edited by multiple users.
- **Exploratory Data Analysis:** They provide a convenient way to perform exploratory data analysis, allowing users to quickly test hypotheses and visualize data1.
- **Integration with Big Data Platforms:** Jupyter Notebooks can integrate with big data processing tools like Apache Spark, enabling the analysis of large datasets.
- **Custom Extensions and Widgets:** The modular design of Jupyter allows for the creation of custom extensions and widgets to enhance functionality and interactivity.

These applications highlight the flexibility and power of Jupyter Notebooks as a tool for interactive computing across a wide range of programming languages and use cases.

## 3.4. Power BI

Power BI is a comprehensive business intelligence platform provided by Microsoft, designed to transform disparate data sources into coherent and visually immersive insights. It's a collection of software services, apps, and connectors that cater to various data analytics needs, from simple data visualization to complex business intelligence tasks. Power BI enables users to connect to a multitude of data sources, whether they are simple Excel spreadsheets or more complex hybrid data warehouses combining cloud-based and on-premises data.

At its core, Power BI consists of three main components: Power BI Desktop, the Power BI service, and Power BI Mobile apps. Power BI Desktop is a Windows desktop application that allows for indepth data analysis and report creation. The Power BI service is an online SaaS solution that facilitates the sharing and consumption of business insights. Meanwhile, Power BI Mobile apps extend the platform's capabilities to iOS, Android, and Windows devices, ensuring access to data insights on the go.

Power BI's versatility is evident in its role-based usage. For instance, a business analyst might use Power BI Desktop extensively to create detailed reports, which are then published to the Power BI service for broader consumption. A sales professional, on the other hand, may primarily utilize the Power BI Mobile app to track sales quotas and delve into sales lead details. Developers can leverage Power BI APIs to push data into models or embed dashboards and reports into custom applications.

The platform also includes Power BI Report Builder for creating paginated reports and Power BI Report Server for publishing reports on-premises. These additional elements ensure that Power BI can serve a wide range of reporting and analytical needs across different organizational roles. Power BI's integration with Microsoft Fabric enhances its capabilities, allowing for seamless collaboration and data-driven decision-making within organizations. Its user-friendly interface and powerful AI Insights feature employ artificial intelligence to uncover patterns and insights within data sets, making it accessible even to those with limited data expertise.

In summary, Power BI stands out as a dynamic and adaptable tool that empowers individuals and organizations to make informed decisions based on data-driven insights. Its comprehensive suite of applications supports a variety of business functions and roles, making it an indispensable asset in the realm of business intelligence.

**Features:**

Power BI is equipped with a robust set of features that cater to diverse data analytics needs. Here are some of the key features:

- **Monthly Product Updates**: Power BI is continuously evolving, with Microsoft adding new features every month. This ensures that users have access to the latest analytics capabilities.
- **Large Dataset Analysis:** Power BI can handle large datasets well beyond the row limits of Excel, allowing for the analysis of datasets containing over 100 million rows.
- **Custom Visualizations:** Users can create custom visualizations using R and Python, which is particularly useful for those who require specialized data representations.
- **Excel Integration:** Power BI offers deep integration with Excel, enabling users to analyse their datasets within the familiar environment of Excel spreadsheets (available in Pro or Premium versions).
- **Geospatial Mapping:** The platform allows for the creation of interactive maps to display data geographically, which can be a powerful tool for spatial analysis.
- **Power Query:** This feature simplifies the process of sourcing and transforming data, making it easier to prepare data for analysis1.
- **Automatic Data Refreshes:** Power BI can be set to automatically refresh data, ensuring that reports and dashboards are always up to date (available in Pro or Premium versions).
- **Power BI Mobile App:** The mobile app extends the functionality of Power BI to mobile devices, allowing users to access insights on the go1.
- **Dataset Reusability:** Users can reuse datasets across different reports and dashboards, which streamlines the report creation process (available in Pro or Premium versions).

- **Microsoft Product Integration:** Power BI integrates seamlessly with other Microsoft products, enhancing its utility within the Microsoft ecosystem.
- These features, among others, make Power BI a versatile and powerful tool for anyone looking to derive meaningful insights from their data.

**Applications**

Power BI is a versatile business intelligence tool that offers a wide range of applications across various industries. Here are some of the key applications of Power BI:

- **Data Visualization:** Power BI excels at turning complex data sets into interactive visualizations, making it easier for users to understand and analyse key data points from various sources in a single dashboard.
- **Real-Time Performance Monitoring:** It allows businesses to monitor their operations in realtime, providing insights into active projects, deadlines, and individual employee performance.
- **Sales Analysis:** Power BI can be used to track user activity, identify sales trends, and understand customer preferences, which can help businesses adjust their strategies to boost sales.
- **Marketing Optimization:** With its robust data visualization capabilities, Power BI can assist in enhancing marketing campaigns by tracking key metrics and developing more effective strategies.
- **Consistent Reporting:** Power BI helps in creating standardized reports, which can be particularly useful for managers who need to derive insights quickly and predict future trends.
- **Server-Level Data Management:** It enables the management of business-related data at the server level, providing a more comprehensive view of information systems.
- **Integration with Internal Software Systems:** Power BI can be integrated with various software platforms used in business operations, such as CRMs, accounting software, and traditional data platforms like Azure and MySQL.
- **Embedding Complex Data:** Power BI can take input from various sources and provide that data within software and apps, enhancing app customization and adding value to products or services.

- **Streamlining Organizational Processes:** By providing clear and actionable insights, Power BI helps in streamlining processes and improving operational efficiency.

These applications demonstrate the flexibility and power of Power BI as a tool for business intelligence, data analysis, and decision-making support.

## 3.5. Data Analysis Expressions (DAX)

Data Analysis Expressions (DAX) is a formula language used in Power BI, Power Pivot for Excel, and SQL Server Analysis Services to extend data models with calculated columns, measures, and tables. DAX formulas are very similar to Excel formulas, but they provide much more power and flexibility to analyse data.

One of the most powerful features of DAX is its ability to create complex calculations, especially aggregations, that can be dynamically adjusted based on user interactions in reports. For example, a measure created in DAX can calculate a sum or average that automatically filters based on what the user selects in a report, without the need for additional programming. DAX also includes a set of time intelligence functions that simplify the process of comparing data across different time periods. This can be incredibly useful for financial and sales analysis, where understanding trends over time is crucial.

Another key aspect of DAX is its use of context to perform calculations. There are two types of contexts: row context and filter context. Row context is used when calculating values for a calculated column, while filter context is applied when evaluating a measure. Understanding and leveraging these contexts is essential for creating accurate and meaningful calculations in Power BI. DAX is not just about creating calculations; it's also a query language that can retrieve data from a model. This is done using the EVALUATE and DEFINE clauses, which specify what data to return and how to calculate it, respectively.

Learning DAX can be challenging, especially for those new to data modelling and analysis. However, once mastered, it provides a powerful toolset for transforming raw data into meaningful insights. Whether you're a business analyst looking to create more dynamic reports, or a data professional needing to perform complex data transformations, DAX is an essential skill in the world of data analytics.

In conclusion, DAX is a versatile and powerful language that is integral to the Microsoft BI stack. Its ability to handle complex calculations, time intelligence, and dynamic aggregation makes it an indispensable tool for anyone working with data in Power BI, Power Pivot, or Analysis Services.

**Features:**

- **Rich Function Library:** DAX provides a rich library of functions that are similar to Excel formulas but are designed to work with relational data and perform dynamic aggregation.
- **Contextual Calculations:** DAX is context-aware, meaning it can perform calculations based on the filters applied to the data. This allows for powerful and dynamic analysis within reports.
- **Time Intelligence:** DAX includes time intelligence functions that make it easy to perform calculations like year-to-date, quarter-to-date, and month-to-date without complex programming.
- **Row Context and Filter Context:** DAX operates under two contexts—row context and filter context. Understanding these contexts is crucial for writing effective DAX formulas1.
- **Calculated Columns and Measures:** DAX can create calculated columns and measures, which are calculations that are added to data models for use in reports and data visualizations.
- **Query Language Capabilities:** DAX also serves as a query language that can retrieve data from a model. It's used in Power BI's DAX query view and other tools like DAX Studio.
- **Integration with SQL Server Management Studio (SSMS):** DAX can be used within SSMS to create and test DAX queries, providing a familiar environment for SQL Server professionals.
- **Simple Syntax:** DAX has a simple syntax with a few key keywords like EVALUATE and DEFINE, making it accessible for users familiar with Excel formulas.
- **Performance Optimization:** DAX is optimized for in-memory computation, which provides fast performance on large datasets.
- **Customization and Flexibility:** DAX formulas can be customized to suit specific business needs, providing flexibility for advanced data modelling.

These features make DAX a versatile and essential tool for anyone working with data in Microsoft's business intelligence platforms.

**Applications**

The Data Analysis Expressions (DAX) language is a powerful tool used in various Microsoft products for data modelling and analysis. Here are some of the primary applications of DAX:

- **Creating Calculated Columns:** DAX is used to add new data to existing tables in your model. For example, you can create a column that is a calculated percentage of two other columns.
- **Developing Measures:** DAX allows you to create measures for dynamic calculations, such as sums or averages, which update as your data changes or as filters are applied within reports.
- **Building Calculated Tables:** You can use DAX to create new tables that are based on data already present in your model. This is useful for creating summaries or aggregations at a different granularity than your source data.
- **Time Intelligence:** DAX provides time intelligence functions that make it easy to perform calculations over time, such as year-to-date, month-to-date, and same-period-last-year comparisons.
- **Filtering and Context Manipulation:** DAX includes functions that allow you to manipulate the context in which a calculation is performed, enabling complex filtering, and slicing of data.
- **Data Analysis:** DAX can be used to analyse data by creating complex formulas that can be used in pivot tables and charts within Power BI, Power Pivot, and Analysis Services.
- **Reporting and Visualization:** DAX formulas can be used to enhance reports and visualizations in Power BI, providing deeper insights into the data through calculated metrics.
- **Row-Level Security:** DAX can be used to implement row-level security in your data models to ensure that users see only the data they are authorized to view.

These applications highlight the versatility of DAX as a language for data analysis and business intelligence, enabling users to derive meaningful insights from their data.

# CHAPTER 4
# System Design

Designing a system for COVID-19 analysis using Power BI involves several steps to ensure that the dashboard is informative, interactive, and user-friendly. Here's a high-level overview of the system design process:

1. **Data Collection**: Identify reliable data sources for COVID-19 statistics. The data should include metrics like confirmed cases, deaths, recoveries, vaccination rates, and population data.
2. **Data Preparation**: Clean and preprocess the data to ensure accuracy. This may involve handling missing values, correcting data types, and removing duplicates.
3. **Data Modelling**: Build a data model in Power BI by importing the data and creating relationships between different tables. This step is crucial for efficient data management and retrieval.
4. **Measure Creation**: Use DAX (Data Analysis Expressions) to create measures that will calculate key performance indicators (KPIs) such as Case Fatality Rate (CFR), Recovery Rate, and Active Cases[1].
5. **Dashboard Design**: Design the dashboard layout with a focus on usability. Include a variety of interactive charts, maps, and tables that allow users to drill down into specific data points and gain deeper insights[2].
6. **Interactivity**: Implement filters and slicers to enable users to customize the view according to date ranges, countries, or regions. This allows for detailed analysis and a personalized experience.
7. **Visualization**: Choose appropriate visualizations for the data. For example, use line charts for trend analysis, maps for geographical distribution, and bar charts for comparing different metrics.
8. **User Feedback**: Test the dashboard with a group of users and gather feedback. Use this feedback to make iterative improvements to the dashboard design and functionality.

9. **Publishing**: Once the dashboard is finalized, publish it to Power BI Service for broader access, or distribute it as a Power BI Desktop file for local use.
10. **Maintenance**: Regularly update the dashboard with the latest data and adjust as needed to reflect the evolving nature of the pandemic.

## 4.1. System Design Flow Chart

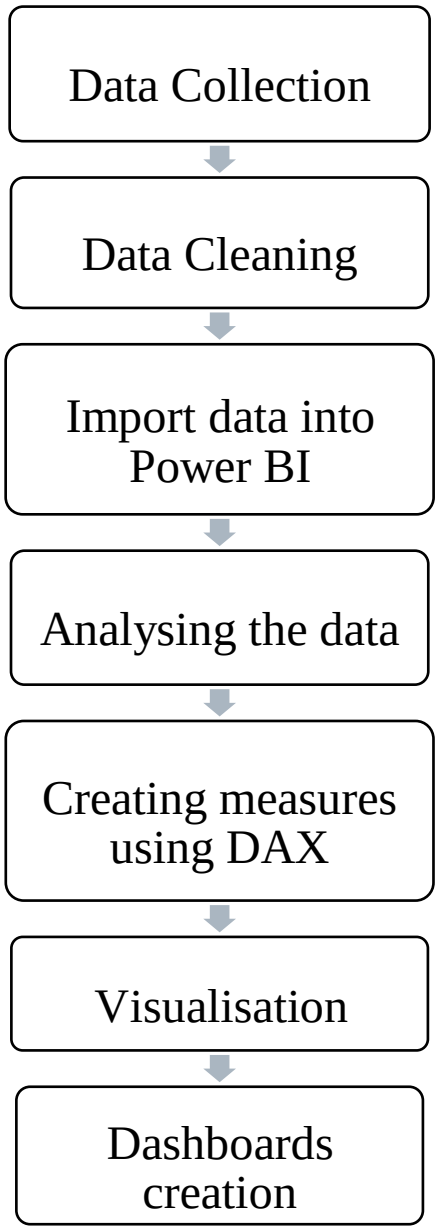



Fig 4.1: System Design Flow Chart

# CHAPTER 5
# Implementation

The aim of the project is to create a report to help Covid Analytics to maintain the records of a large data of cases, handle cases details, and understand how the cases increased. Covid analytics deals with the maintenance of cases and mortality rate in India as well as globally. This Covid analytics system is user friendly. Having high performance and no time consuming.

The main objective in making this dashboard is that the reader can directly analyse what we had in past years records, which areas are having high cases, or which is having high recoveries, etc.

## 5.1. Data Source

Retail raw data taken from google.com.

There are three sheets:

a. time_series_covid19_confirmed_global

```
raw_data_confirmed.head()
```

| | Province/State | Country/Region | Lat | Long | 1/22/20 | 1/23/20 | 1/24/20 | 1/25/20 | 1/26/20 | 1/27/20 | ... | 12/23/20 | 12/24/20 | 12/25/20 | 12/26/20 | 12/27/20 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | NaN | Afghanistan | 33.93911 | 67.709953 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 50433 | 50655 | 50810 | 50886 | 51039 | |
| 1 | NaN | Albania | 41.15330 | 20.168300 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 54317 | 54827 | 55380 | 55755 | 56254 | |
| 2 | NaN | Algeria | 28.03390 | 1.659600 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 96549 | 97007 | 97441 | 97857 | 98249 | |
| 3 | NaN | Andorra | 42.50630 | 1.521800 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 7669 | 7699 | 7756 | 7806 | 7821 | |
| 4 | NaN | Angola | -11.20270 | 17.873900 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 16931 | 17029 | 17099 | 17149 | 17240 | |

5 rows × 350 columns

Fig 5.1: Head of confirmed global data

b. time_series_covid19_recovered_global

```
raw_data_Recovered.head()
```

| | Province/State | Country/Region | Lat | Long | 1/22/20 | 1/23/20 | 1/24/20 | 1/25/20 | 1/26/20 | 1/27/20 | ... | 12/23/20 | 12/24/20 | 12/25/20 | 12/26/20 | 12/27/20 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | NaN | Afghanistan | 33.93911 | 67.709953 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 39692 | 40359 | 40444 | 40784 | 41096 | |
| 1 | NaN | Albania | 41.15330 | 20.168300 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 29799 | 30276 | 30790 | 31181 | 31565 | |
| 2 | NaN | Algeria | 28.03390 | 1.659600 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 64401 | 64777 | 65144 | 65505 | 65862 | |
| 3 | NaN | Andorra | 42.50630 | 1.521800 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 7106 | 7171 | 7203 | 7252 | 7288 | |
| 4 | NaN | Angola | -11.20270 | 17.873900 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 9729 | 9729 | 9921 | 9976 | 10354 | |

5 rows × 350 columns

Fig 5.2: Head of recovered global data

c. time_series_covid19_deaths_global

```
raw_data_deaths.head()
```

| | Province/State | Country/Region | Lat | Long | 1/22/20 | 1/23/20 | 1/24/20 | 1/25/20 | 1/26/20 | 1/27/20 | ... | 12/23/20 | 12/24/20 | 12/25/20 | 12/26/20 | 12/27/20 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | NaN | Afghanistan | 33.93911 | 67.709953 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 2117 | 2126 | 2139 | 2149 | 2160 | |
| 1 | NaN | Albania | 41.15330 | 20.168300 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 1117 | 1125 | 1134 | 1143 | 1153 | |
| 2 | NaN | Algeria | 28.03390 | 1.659600 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 2696 | 2705 | 2716 | 2722 | 2728 | |
| 3 | NaN | Andorra | 42.50630 | 1.521800 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 82 | 83 | 83 | 83 | 83 | |
| 4 | NaN | Angola | -11.20270 | 17.873900 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 393 | 393 | 396 | 399 | 399 | |

5 rows × 350 columns

Fig 5.3: Head of deaths global data

## 5.2 Data Cleaning

Data cleaning is an essential process in data analysis, especially in Python, where libraries like Pandas and NumPy make it a manageable task. The process involves several steps to ensure the data is accurate, consistent, and ready for analysis. It starts with removing duplicates to prevent skewed results

and continues with handling missing values, which can be filled with statistical measures like mean or median, or removed entirely depending on the scenario.

Data type conversion is another critical step, ensuring that numerical data is not mistakenly processed as strings and dates are in the correct format for time-series analysis. Outlier detection is also crucial; outliers can be legitimate but can also indicate errors or noise in the data. They can be managed by methods like trimming, capping, or transformation.

Normalization and standardization of data are important for models that are sensitive to the scale of data, like k-means clustering or k-nearest neighbours. Encoding categorical variables into numerical values is necessary for machine learning algorithms to interpret the data correctly.
Techniques like one-hot encoding or label encoding are commonly used.

Error correction involves fixing typos or inconsistent capitalization, which is particularly important for textual data. Regular expressions and string operations in Python are powerful tools for such tasks Data validation ensures that the data meets certain criteria, such as constraints on values or formats, which can be enforced using custom functions or Pandas' validation features.

Finally, feature engineering can be considered part of data cleaning, where new features are created from existing data to improve the model's performance. This could involve creating polynomial features, interaction terms, or aggregating data to a higher level.
In summary, data cleaning in Python is a multifaceted process that improves the quality of data and is a prerequisite for any reliable data analysis. It's a combination of domain knowledge, statistical understanding, and programming skills that together form the foundation of any datadriven project.

### 5.2.1 UN-pivoting data

In Python, you can **unpivot** or **melt** a DataFrame using the pandas library. Unpivoting involves converting data from a **wide format** (where columns represent different variables) to a **long format** (where rows represent observations).

**Un-Pivoting the data**

```
raw_data_confirmed2 = pd.melt(raw_data_confirmed, id_vars=['Province/State', 'Country/Region', 'Lat', 'Long'],
                              var_name=['Date'])
raw_data_deaths2 = pd.melt(raw_data_deaths, id_vars=['Province/State', 'Country/Region', 'Lat', 'Long'],
                           var_name=['Date'])
raw_data_Recovered2 = pd.melt(raw_data_Recovered, id_vars=['Province/State', 'Country/Region', 'Lat', 'Long'],
                              var_name=['Date'])


print("The Shape of Confirmed is: ", raw_data_confirmed2.shape)
print("The Shape of deaths is: ", raw_data_deaths2.shape)
print("The Shape of recovered is: ", raw_data_Recovered2.shape)


raw_data_confirmed2
```

Fig 5.4: Un-pivoting data

### 5.2.2 Investigating Null Values

Investigating and handling null values in a dataset is a crucial aspect of data preprocessing, which can be efficiently performed using Python's Pandas library. The process begins with the identification of null values using the isnull() function, which returns a boolean mask indicating the presence of missing data. Once identified, analysts can count the number of nulls in each column using the sum() method on the boolean mask. For a more granular approach, the np.where() function can be employed to pinpoint the exact locations of these null values within the DataFrame.

After locating the null values, one might choose to filter out rows that contain them, especially if they pertain to a specific column of interest. This is achieved through boolean indexing. Alternatively, null values can be replaced with a specified value or a statistical measure such as the mean or median of the column, using the fillna() method. In cases where the presence of null

values is not acceptable, the dropna() method allows for the removal of rows or columns that contain them.

In **Pandas**, you can check for null values using the following methods:

1. isnull():

The isnull() method returns a DataFrame of Boolean values where **True** indicates the presence of **NaN** (Not a Number) values in the specified DataFrame.

2. notnull():

The notnull() method returns the opposite of isnull(), i.e., True for non-null values and False for null values.

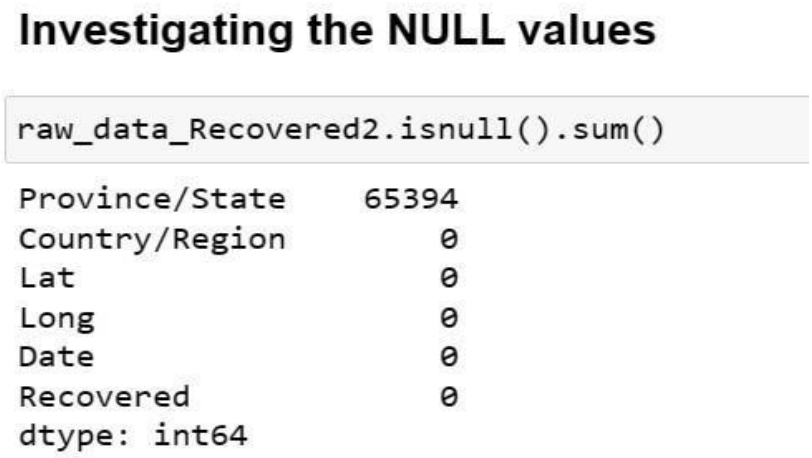



Fig 5.5: Investigating null values

### 5.2.3 Dealing with Null values

Dealing with null values is a common task in data cleaning and preprocessing. In Pandas, null values can be handled in several ways, depending on the context and the desired outcome.

In Python, the fillna() method is a powerful tool for handling missing or null values in **Pandas DataFrames**.

1. **Syntax**:

- The fillna() method replaces **NaN** (Not a Number) values with a specified value.
- It can be applied to a DataFrame or a Series.

2. **Parameters**:

- value: The value to use for filling holes (default is **None**).
- method: Method to use for filling holes (e.g., **'ffill'** for forward fill, **'bfill'** for backward fill).
  - axis: Specify the axis along which filling should occur (0 for rows, 1 for columns).
- inplace: If **True**, fill in place (modify the original DataFrame). ○ limit: Maximum number of consecutive NaN values to forward/backward fill. ○ downcast: Specify data type for downcasting (e.g., **'integer'**, **'float'**).

**Dealing with NULL values**

```
raw_data_confirmed2['Province/State'].fillna(raw_data_confirmed2['Country/Region'], inplace=True)
raw_data_deaths2['Province/State'].fillna(raw_data_deaths2['Country/Region'], inplace=True)
raw_data_Recovered2['Province/State'].fillna(raw_data_Recovered2['Country/Region'], inplace=True)
```

```
raw_data_Recovered2.isnull().sum()
```

```
Province/State    0
Country/Region    0
Lat               0
Long              0
Date              0
Recovered         0
dtype: int64
```

Fig 5.6: Dealing with null values

## 5.3 How to upload raw data into Power BI

To Create Power BI, visualize dashboard, you need to download Power bi desktop and install. Once you have installed, then we need to follow below process (Microsoft Power BI documentation, 2021):

Click on Get Data => Excel workbook = > browse your raw file = > select required sheets and then click on transform => Power Query editor window will be open => then from each table you have to check all data types, then you have to click on close & apply.

Now your base data is ready!

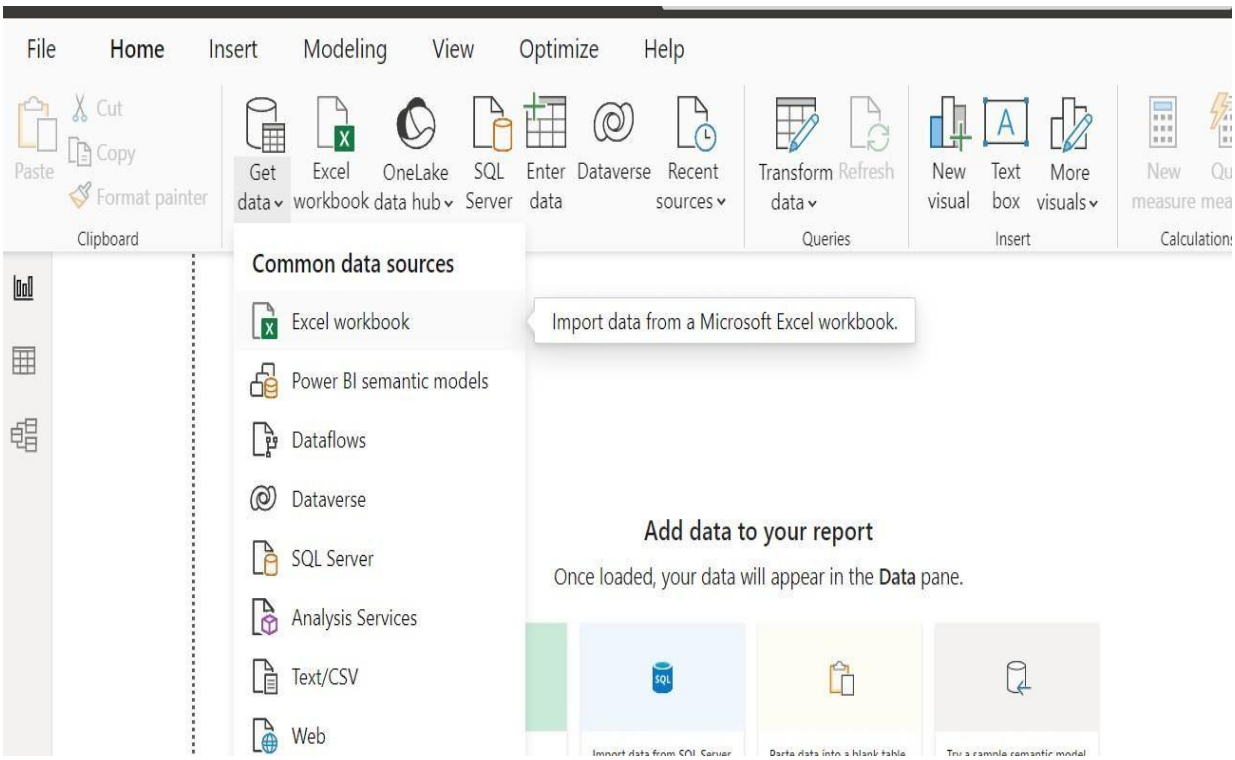



Fig 5.7: Upload data into Power BI

## 5.4. DAX Queries

Creating measures in DAX (Data Analysis Expressions) is a fundamental part of working with Power BI to perform dynamic calculations. Steps to create measures using DAX are:

1. **Open Power BI Desktop** and import data from your data source[1].
2. **Select the table** where you want to create the measure.
3. **Click on the 'New Measure'** button at the top of the screen.
4. **Type a name for your measure** in the formula bar.
5. **Enter the DAX formula** for the measure.

I have created multiple measures with DAX query. Some of them are:

**1) Active Cases:** to daily active cases & below is the DAX query.

Active Cases = SUM('CoronaVirus Data'[Confirmed Daily])-(SUM('CoronaVirus Data'[Deaths Daily])+SUM('CoronaVirus Data'[Recovered Daily]))

**2) Average Confirm daily**: to calculate average confirm daily cases & below is the DAX query.

Average Confirm daily = SUM('CoronaVirus Data'[Confirmed Daily]) / DISTINCTCOUNT('CoronaVirus Data'[Date])

**3) Average Deaths daily:** to calculate average confirm daily cases & below is the DAX query.

Average Deaths daily = SUM('CoronaVirus Data'[Deaths Daily]) / DISTINCTCOUNT('CoronaVirus Data'[Date])

**4) Average Recoveries daily:** to calculate average confirm daily cases & below is the DAX query.

Average Recoveries daily= SUM('CoronaVirus Data'[Recovered Daily]) / DISTINCTCOUNT('CoronaVirus Data'[Date])

# CHAPTER 6 Data Visualization

In Power BI, data visualization is a fundamental aspect that empowers users to create insightful reports and dashboards. Here is how the original text aligns with Power BI:

1. **Data Exploration and Transformation**: Power BI allows you to connect to various data sources, transform raw data, and create data models. Just like data visualization enhances understanding, Power BI's data transformation capabilities enable you to clean, shape, and enrich your data for better insights.
2. **Visualizations**: In Power BI, you can create a wide range of visualizations, including charts (such as bar charts, line charts, and scatter plots), tables, matrices, maps, and custom visuals. These visual elements help convey complex information in an intuitive manner.
3. **Dashboards and Reports**: Power BI enables you to build interactive dashboards and reports. Dashboards provide a high-level overview, while reports offer detailed analysis. You can pin visualizations to dashboards, filter data, and drill down into specific details.
4. **Data Relationships**: Similar to the goal of distilling large datasets into visual graphics, Power BI emphasizes establishing relationships between different data tables. By defining relationships, you can create meaningful visualizations that showcase connections within the data.
5. **Interactivity**: Power BI visualizations are interactive. Users can explore data by clicking on elements, applying filters, and interacting with slicers. This interactivity enhances engagement and allows users to uncover hidden insights.
6. **Publishing and Sharing**: Once you've created compelling visualizations, you can publish your Power BI reports to the Power BI service. From there, you can share them with colleagues, stakeholders, or clients. Collaboration and sharing are essential aspects of data visualization in Power BI.

## 6.1 Need of Data Visualization

**Data Visualization** is an indispensable tool in the modern data-driven landscape. Let's explore why it's so crucial:

1. **Managing Data Overload**: As the World Economic Forum highlights, we're inundated with an astounding amount of data—2.6 quintillion bytes daily! With this deluge, sifting through raw

   data line-by-line becomes impractical. Data visualization acts as a lifeline, allowing us to distill complex information into visual summaries. Our brains are wired to process visuals more efficiently than rows of text in a spreadsheet.

2. **Spotting Trends and Patterns**: Imagine analyzing years of company profits. A line chart instantly reveals the upward trajectory, interrupted only in 2018. This visual insight—profits consistently rising except for that one dip—would take much longer to glean from a table. Whether it's line charts, bar graphs, or scatter plots, data visualization uncovers trends that might otherwise remain hidden.

3. **Enhancing Insights**: Data gains value when visualized. Even if a skilled data analyst can extract insights without visuals, conveying meaning becomes challenging. Charts and graphs bridge this gap, making data findings more accessible. For instance, a Tableau visualization shows sales by customer—red for losses, grey for profits—highlighting the paradox of high sales yet financial setbacks.

4. **Types of Visualizations**: The world of data visualization is rich and diverse. Here are some common types:

- **Line Charts**: Ideal for tracking trends over time.
- **Box Plots**: Uncover distribution and outliers.
- **Area Charts**: Visualize cumulative data.
- **Sankey Diagrams**: Depict flow and relationships.
- **Scatter Plots**: Show correlations.
- **Bar Charts**: Compare categories.

- **Population Pyramids**: Display age and gender distributions.
- **Pie Charts**: Represent proportions.
- **Heat Maps**: Depict intensity or concentration.
- **Tree Maps**: Visualize hierarchical data.
- **Histograms**: Illustrate frequency distributions.
- **Bubble Charts**: Combine data dimensions.
- **Choropleth Maps**: Map data by geographic regions.
- **Network Diagrams**: Reveal connections.

## 6.2 Plotting data:

Data is plotted into several types of graphs in Power BI. According to the need, different visualisation will be made.

### 6.2.1. Global Data Visualisation

- **Graph of no. of cases in different category (total, active, recovery, and death)**

In this graph, it is shown that what are the daily total cases with no. of active, no. of recovered, and no. of died on daily basis.

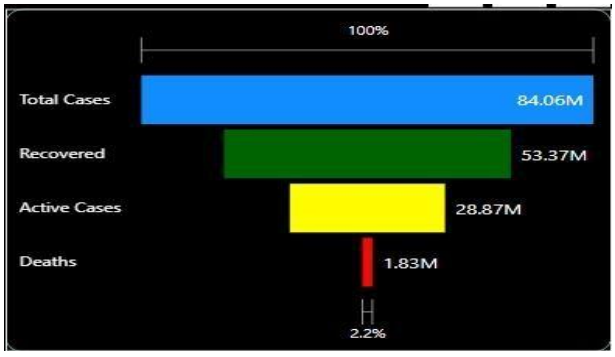



Fig 6.1 Funnel chart (Global)

- **Graph of confirmed, recovered and death cases vs date**

It is shown in the graph that how the confirmed, recovered and deaths cases increase with each date.

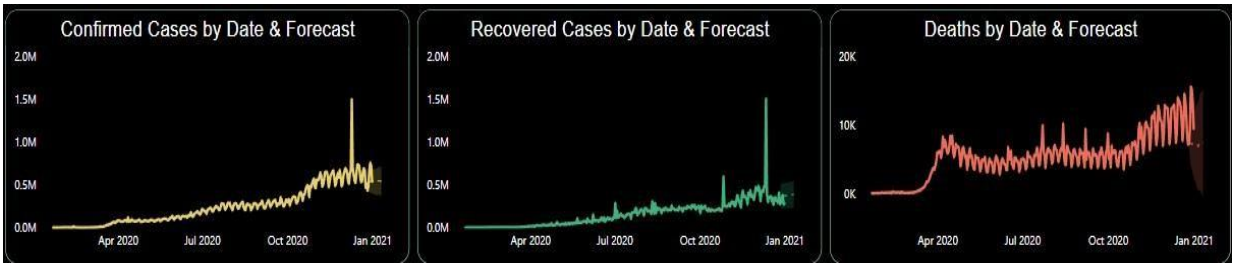



Fig 6.2 Line chart of confirmed, recovered and deaths daily (Global)

- **Graph of cumulative increase in daily cases**

In this graph, it is shown that how values cumulatively increase with each date in confirmed, recovered, and death cases.

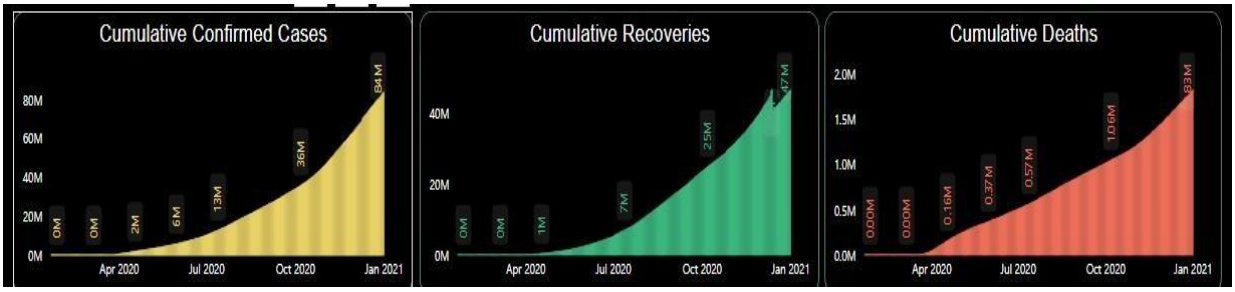



Fig 6.3 Clustered Column Chart (Global)

- **Graph of Confirmed cases by country/region**

It shows the graph for the confirmed cases for each country with its values.

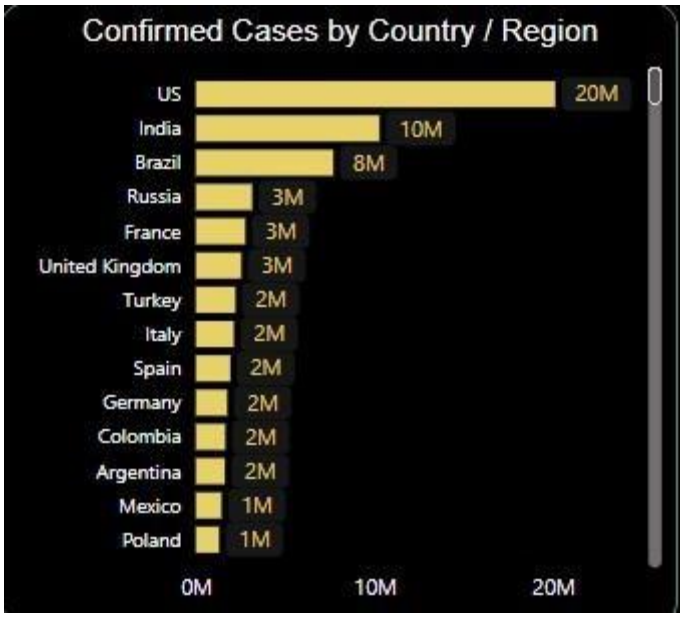



Fig 6.4 Clustered Bar Chart (Global)

- **Graph of cumulative confirmed, recovery, deaths by date**

This graph shows us simultaneously the analysis of confirmed, recovered, and death cases by date.

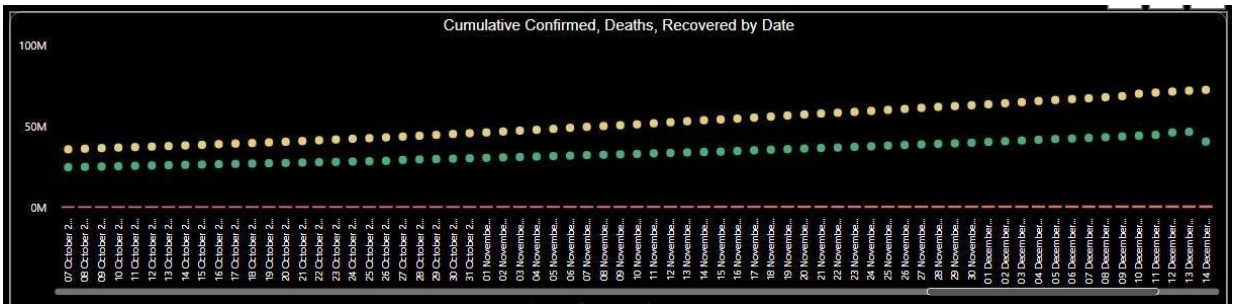



Fig 6.5 Line and clustered column chart (Global)

- **Slicers**

Slicers are added for filtering the data according to the need of the user.

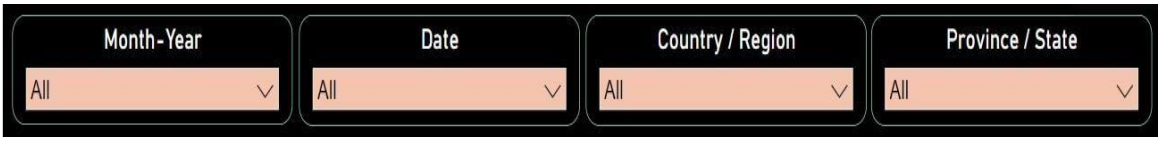



Fig 6.6: Slicer for Global dashboard

### 6.2.2. India Data Visualisation

- **Grape on cumulative confirmed, recovery, deaths by date**

This graph shows us simultaneously the analysis of confirmed, recovered, and death cases by date.

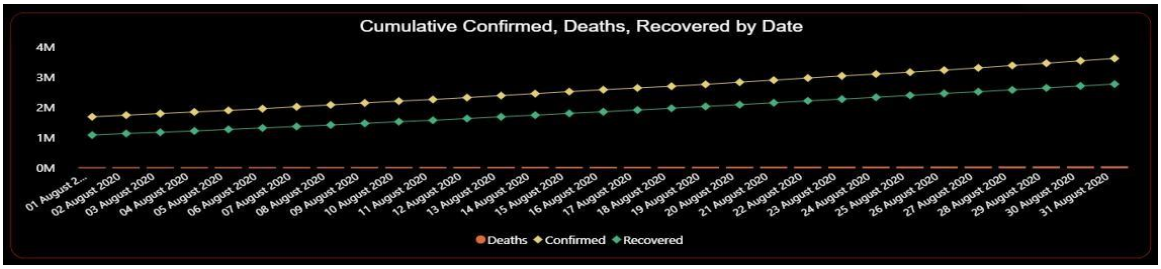



Fig 6.7: Line and clustered column chart (India)

**Graph of covid confirmed cases in India on geographical map**

This graph shows us simultaneously the analysis of confirmed cases by date on geographical map of India.

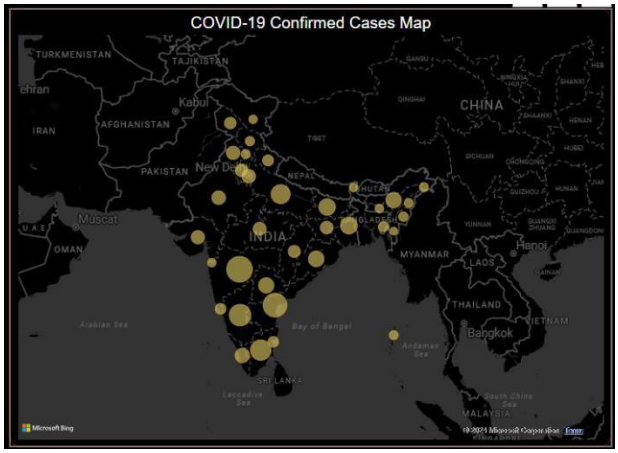



Fig 6.8: Confirmed cases Map (India)

- **Graph of Confirmed cases by country/region**

It shows the graph for the confirmed cases for India country with its values.

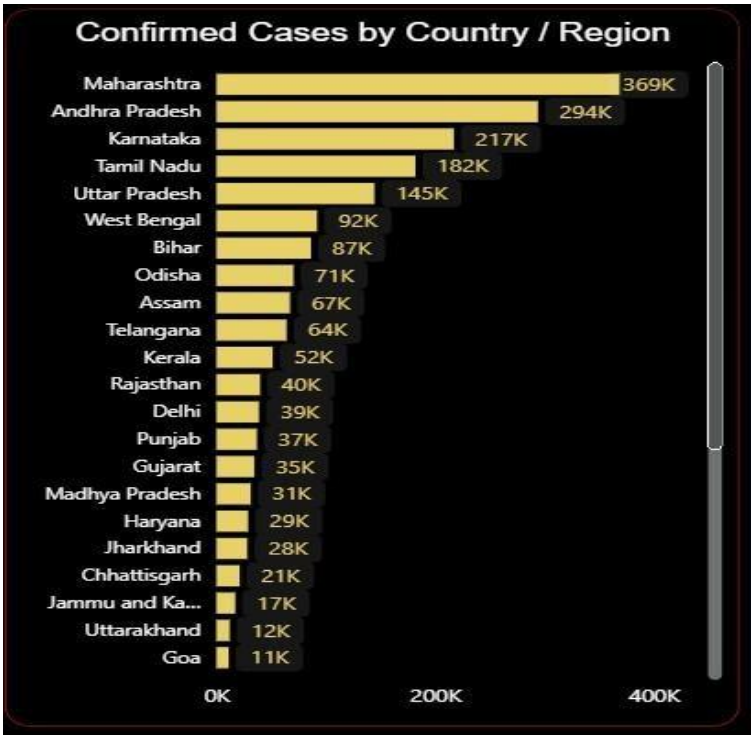



Fig 6.9: Clustered Bar Chart (India)

- **Table having different columns**

The given table has different columns from Indian data. The columns are:

1. Total confirmed cases
2. Confirmed case percentage
3. Total deaths
4. Mortality rate
5. Total recovered cases
6. Recovery rate
7. Average confirmed daily
8. Average recoveries daily
9. Average deaths daily

It explains the complete Indian data in table format.

Province / State Breakdown

| State/UnionTerritory | Total Confirmed | Confirmed % of Total | Total Deaths | Mortality Rate | Total Recovered | Recovery Rate | Average Confirm Daily | Average Recoveries Daily | Average Deaths Daily |
|---|---|---|---|---|---|---|---|---|---|
| Maharashtra | 3,68,891 | 18.61% | 9,670 | 2.62% | 3,13,786 | 85.06% | 11,900 | 10,122 | 312 |
| Andhra Pradesh | 2,94,210 | 14.84% | 2,603 | 0.88% | 2,61,730 | 88.96% | 9,491 | 8,443 | 84 |
| Karnataka | 2,17,296 | 10.96% | 3,359 | 1.55% | 1,95,535 | 89.99% | 7,010 | 6,308 | 108 |
| Tamil Nadu | 1,82,107 | 9.19% | 3,393 | 1.86% | 1,83,955 | 101.01% | 5,874 | 5,934 | 109 |
| Uttar Pradesh | 1,44,593 | 7.29% | 1,836 | 1.27% | 1,20,740 | 83.50% | 4,664 | 3,895 | 59 |
| West Bengal | 92,093 | 4.65% | 1,640 | 1.78% | 84,696 | 91.97% | 2,971 | 2,732 | 53 |
| Bihar | 86,558 | 4.37% | 296 | 0.34% | 85,774 | 99.09% | 2,792 | 2,767 | 10 |
| Odisha | 70,556 | 3.56% | 313 | 0.44% | 53,487 | 75.81% | 2,276 | 1,725 | 10 |
| Assam | 67,367 | 3.40% | 202 | 0.30% | 54,847 | 81.42% | 2,173 | 1,769 | 7 |
| Telangana | 64,246 | 3.24% | 322 | 0.50% | 48,265 | 75.13% | 2,072 | 1,557 | 10 |
| Kerala | 51,552 | 2.60% | 217 | 0.42% | 37,690 | 73.11% | 1,663 | 1,216 | 7 |
| Rajasthan | 40,082 | 2.02% | 380 | 0.95% | 36,708 | 91.58% | 1,293 | 1,184 | 12 |
| Delhi | 38,987 | 1.97% | 490 | 1.26% | 34,447 | 88.36% | 1,258 | 1,111 | 16 |
| **Total** | **19,82,375** | **100.00%** | **28,722** | **1.45%** | **17,16,996** | **86.61%** | **63,948** | **55,387** | **927** |

Fig 6.10: Table (India)

- **Graph of confirmed, recovered and death cases vs date**

It is shown in the graph that how the confirmed, recovered and deaths cases increase with each date.


Fig 6.11: Line chart of confirmed, recovered and deaths daily (India)

- **Graph of cumulative increase in daily cases**

In this graph, it is shown that how values cumulatively increase with each date in confirmed, recovered, and death cases.

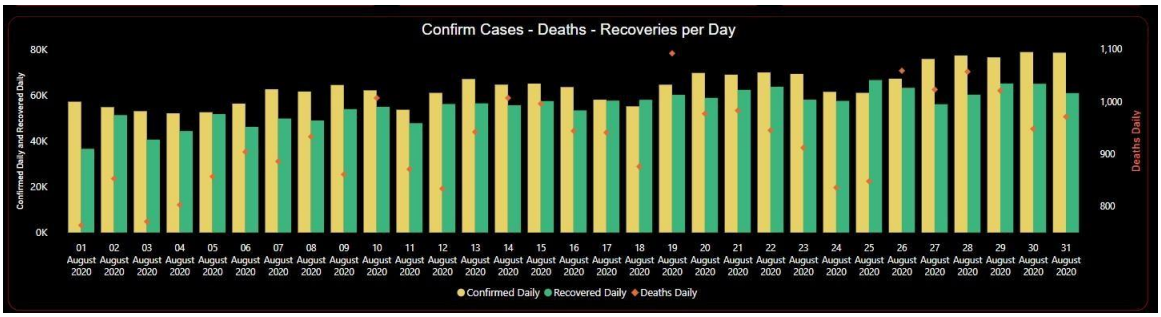


Fig 6.12: Clustered Column Chart (India)

- **Slicers**

Slicers are added for filtering the data according to the need of the user.


Fig 6.13: Slicer for India dashboard

## 6.3 Creating Dashboards in Power BI

For creating a dashboard in Power BI, various visualisations are combined in a report. Preparing a dashboard in Power BI involves several key steps to transform your data into a visual and interactive dashboard. Here's a high-level overview of the process:

1. **Connect to Data Sources**: Begin by connecting Power BI to various data sources such as Excel spreadsheets, databases, or cloud services.
2. **Manage Data Model**: Organize your data by creating relationships between different tables and preparing your data model for analysis.
3. **Calculate Measures**: Use DAX (Data Analysis Expressions) to calculate the metrics you'll need for your dashboard.
4. **Create Visualizations**: Select from a range of visualizations like charts, graphs, and maps to represent your data.
5. **Arrange Dashboard**: Organize your visualizations logically on the dashboard, grouping related data together.
6. **Enhance Interactivity**: Add features like tooltips, drill-downs, and filters to make your dashboard interactive.
7. **Keep It Simple**: Aim for a clean and uncluttered dashboard design to make it easy to understand.

## Initial Dashboard

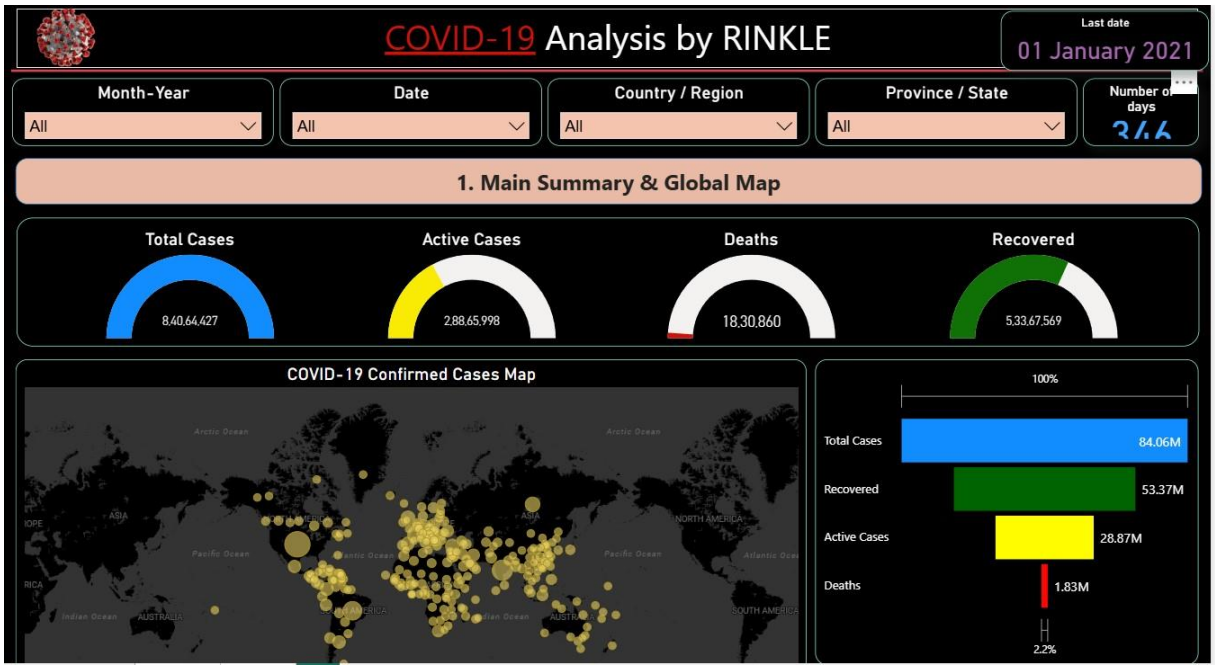



Fig 6.14: Initial Dashboard

# CHAPTER 7
# Testing

## 7.1. Test Cases and Results

- **Test Case 1**

Test Case Id: UU01

Test scenario: Show data of month April,2020 for all dates and countries.

Expected result: Data of April,2020 for all is shown.

Actual result: As expected.

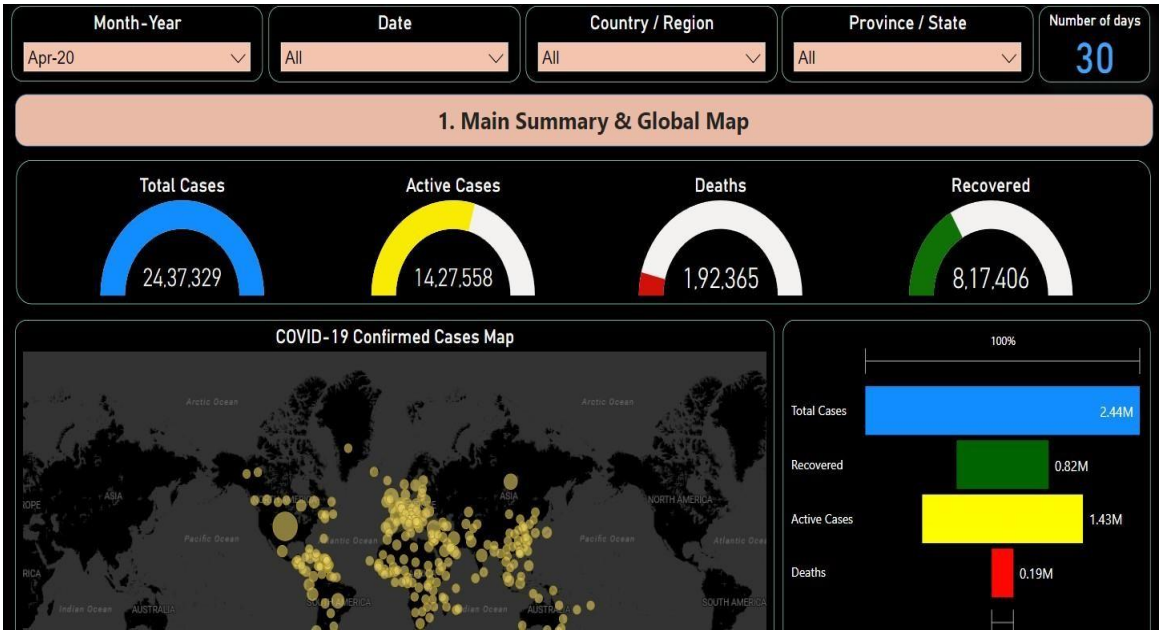



Fig 7.1: Test cases 1

- **Test Case 2**

Test Case Id: UU02

Test scenario: Show data of month April,2020 for all dates for China only.

Expected result: Data of April,2020 for China only is shown.

Actual result: As expected.

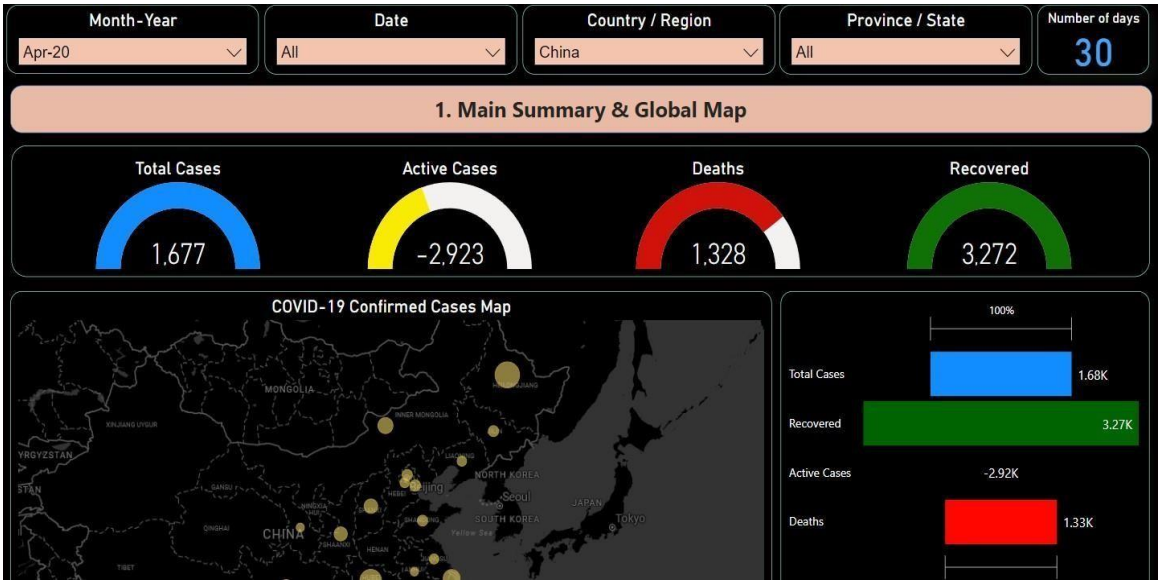



Fig 7.2: Test case 2

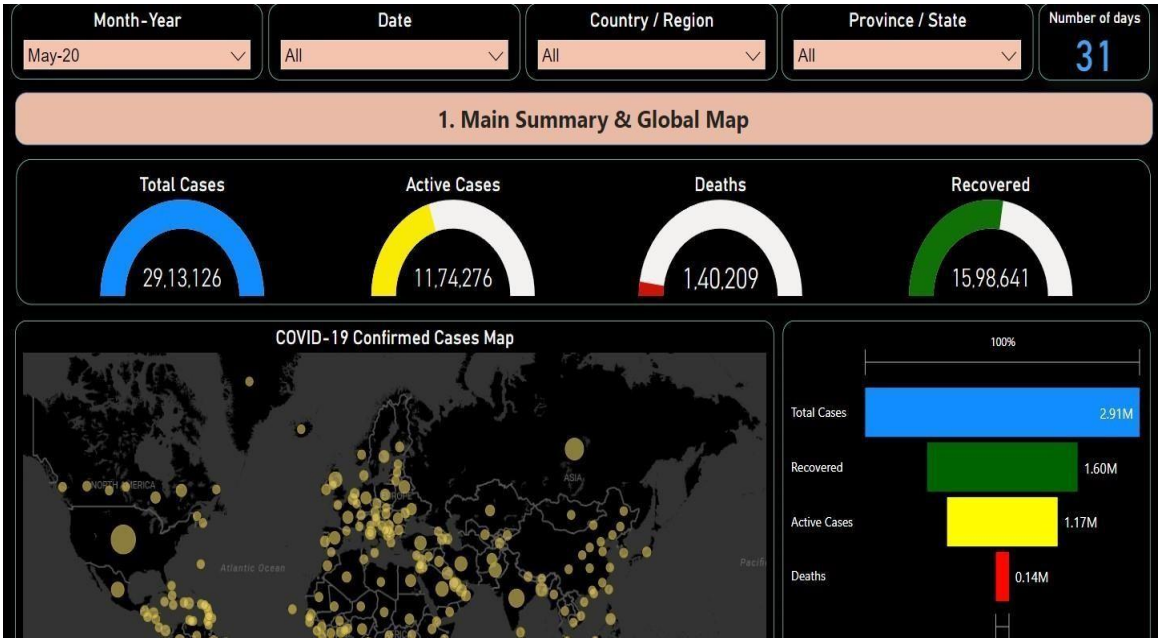
- **Test Case 3**

Test Case Id: UU03

Test scenario: Show data of month May 2020 for all dates for all countries.

Expected result: Data of May 2020 for all dates for all is shown.

Actual result: As expected.



- **Test Case 4**

Test Case Id: UU04

Test scenario: Show data of month May 2020 for all dates for China Only.

Expected result: Data of May 2020 for all dates for China only is shown.

Actual result: As expected.

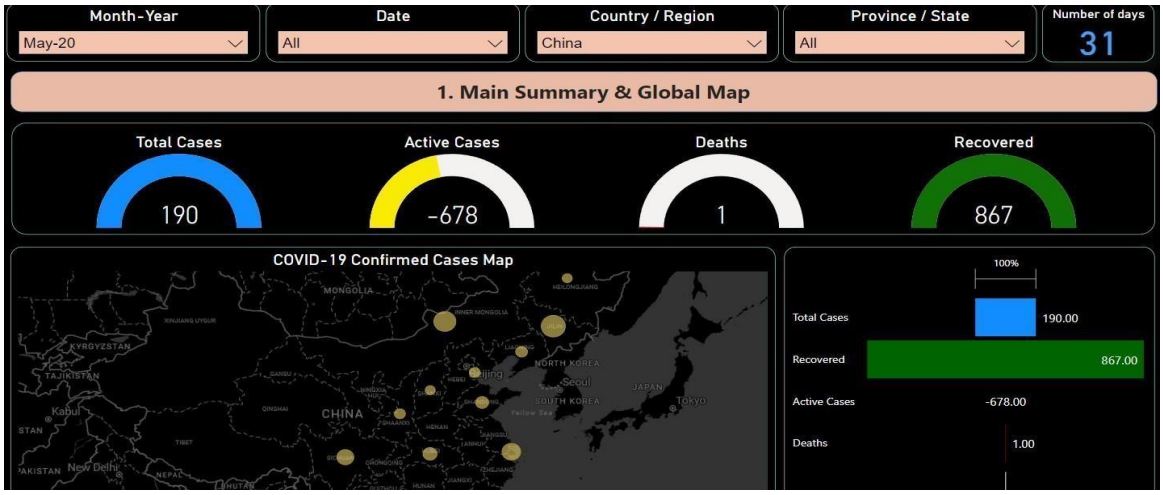



Fig 7.4: Test case 4

- **Test Case 5**

Test Case Id: UU05

Test scenario: Show data of month April,2020 for all dates for India.

Expected result: Data of April,2020 for India is shown.

Actual result: As expected.

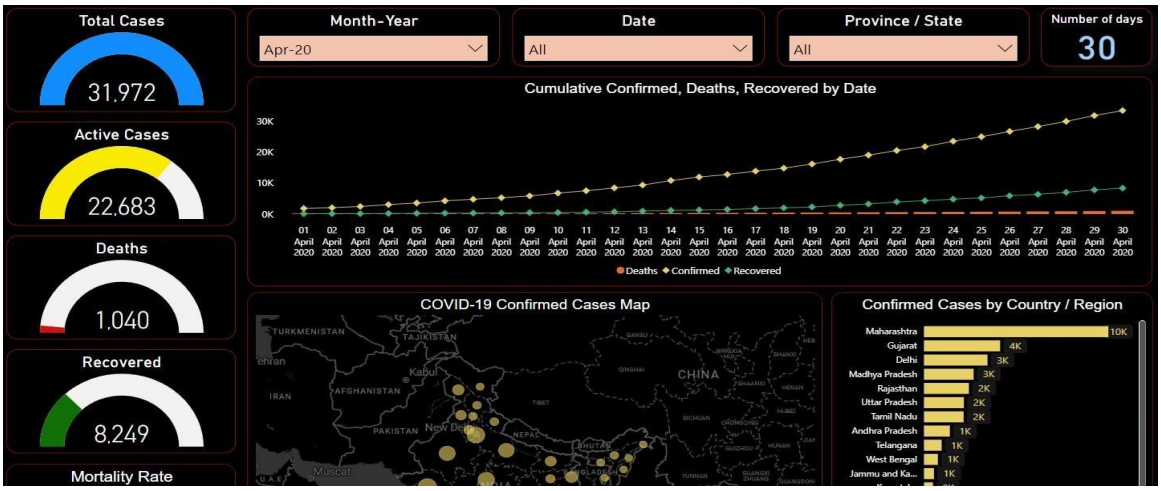



Fig 7.5: Test case 5

- **Test Case 6**

Test Case Id: UU06

Test scenario: Show data of month April,2020 for all dates for India for Maharashtra.

Expected result: Data of April,2020 for India for Maharashtra is shown.

Actual result: As expected.

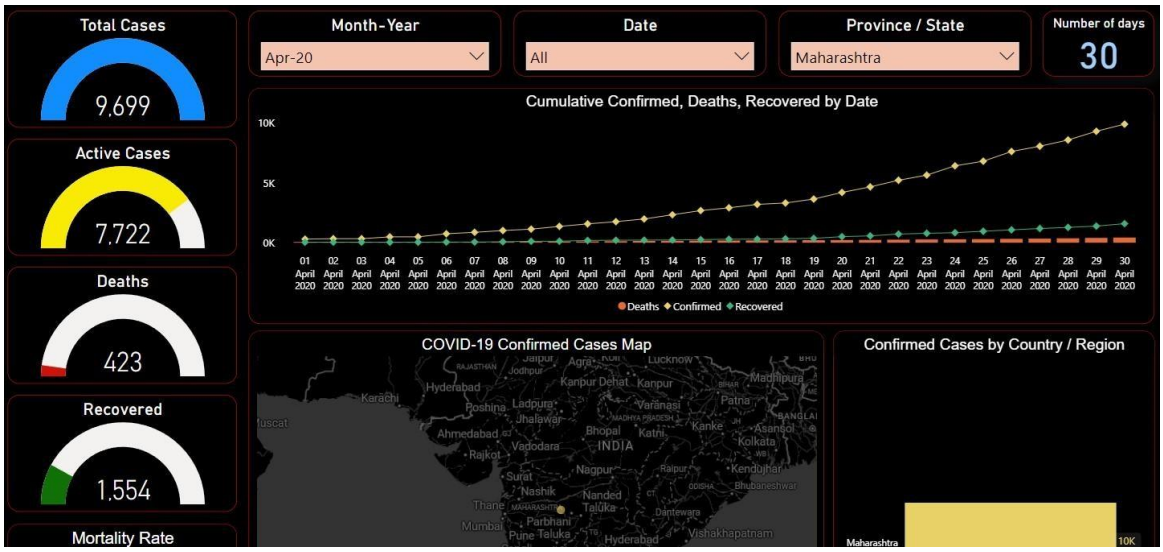



Fig 7.6: Test case 6

7.2. Test Cases Table

Table 1- Test Case

| Test Case ID | Test Scenario | Expected Result | Actual Result |
|---|---|---|---|
| UU01 | Show data of month April,2020 for all dates and countries. | Data of April,2020 for all is shown. | As expected. |
| UU02 | Data of April 2020 for all dates for China only is shown. | Data of April,2020 for China only is shown. | As expected. |
| UU03 | Show data of month May,2020 for all dates and countries. | Data of May,2020 for all is shown. | As expected. |
| UU04 | Data of May 2020 for all dates for China only is shown. | Data of May,2020 for China only is shown. | As expected. |
| UU05 | Show data of month April,2020 for all dates for India | Data of April,2020 for India | As expected. |
| UU06 | Show data of month April,2020 for all dates for India for Maharashtra. | Data of April,2020 for India for Maharashtra is shown. | As expected. |

# CHAPTER 8 Dashboard Reports

## 8.1. Global Data Dashboard

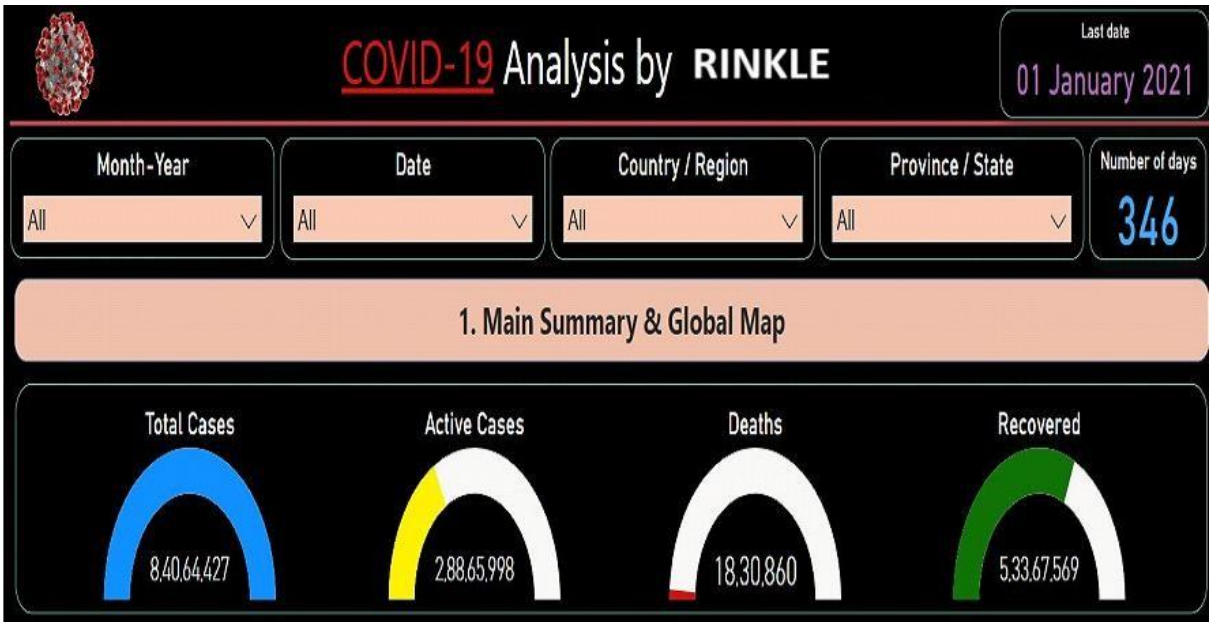



Fig 8.1: Global Data Dashboard (1)

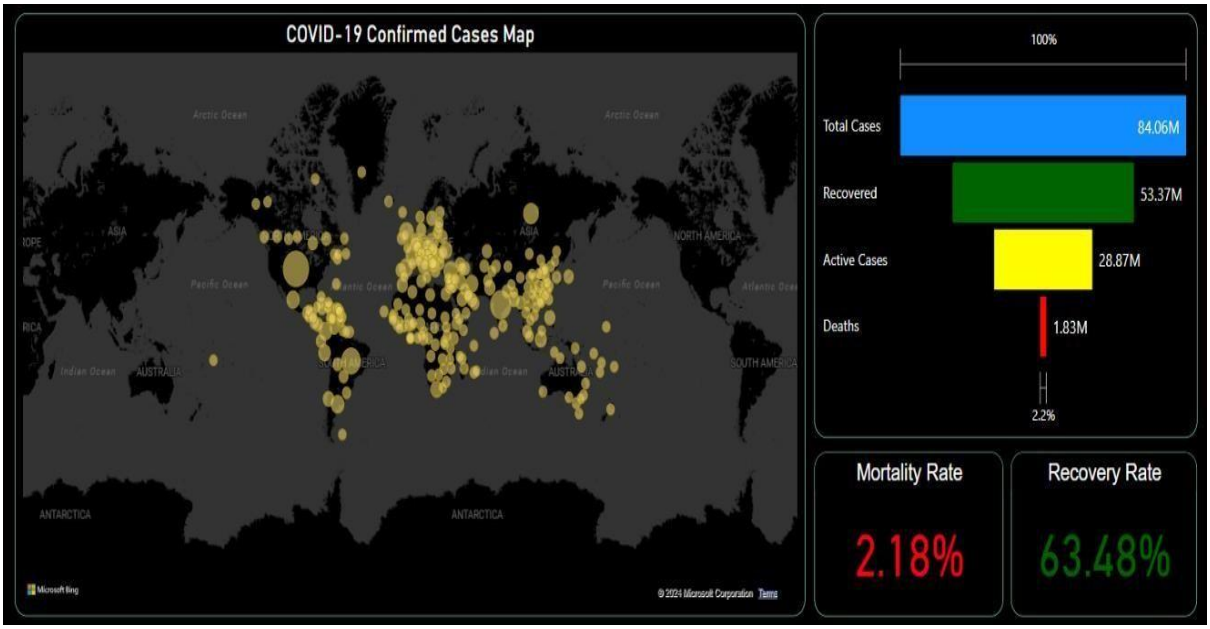



Fig 8.2: Global Data Dashboard (2)

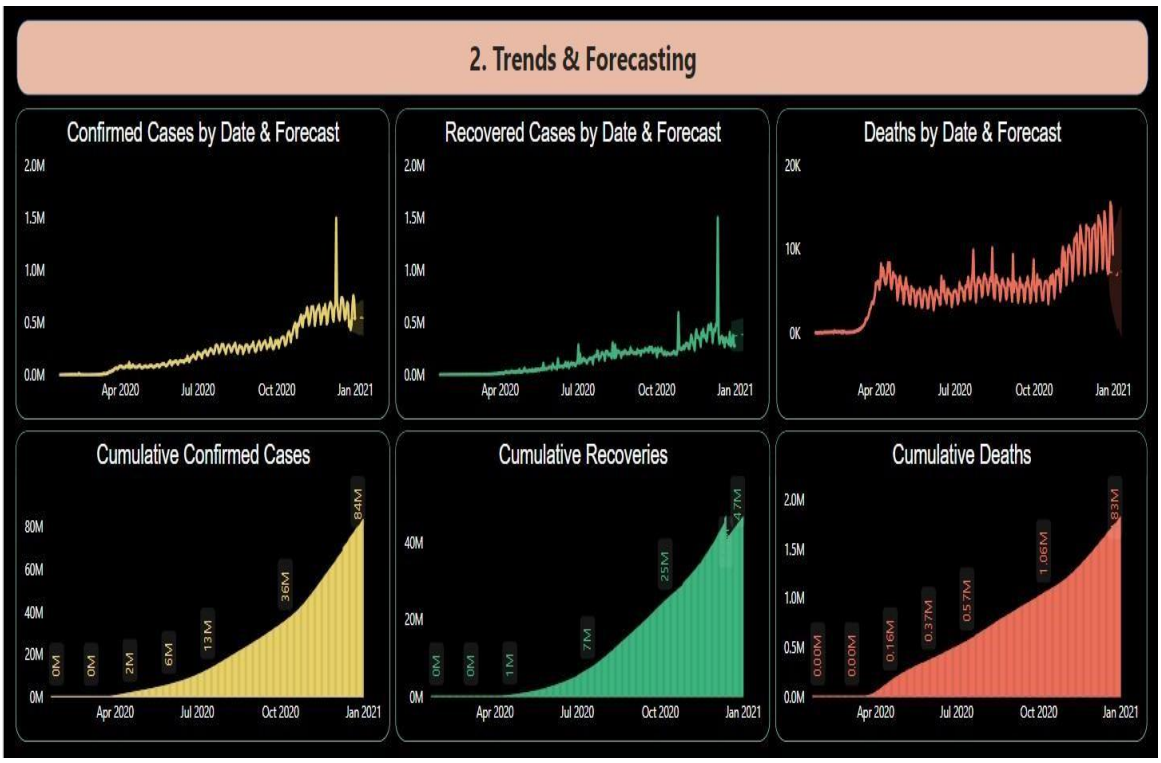



Fig 8.2: Global Data Dashboard (3)

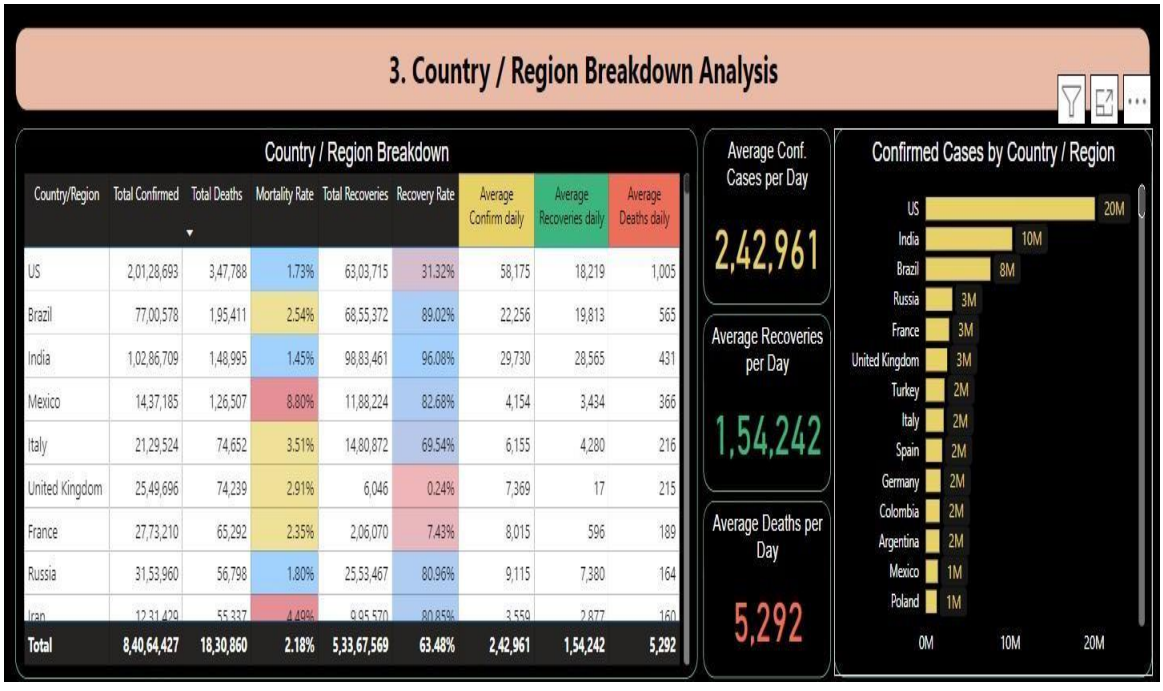



Fig 8.4: Global Data Dashboard (4)

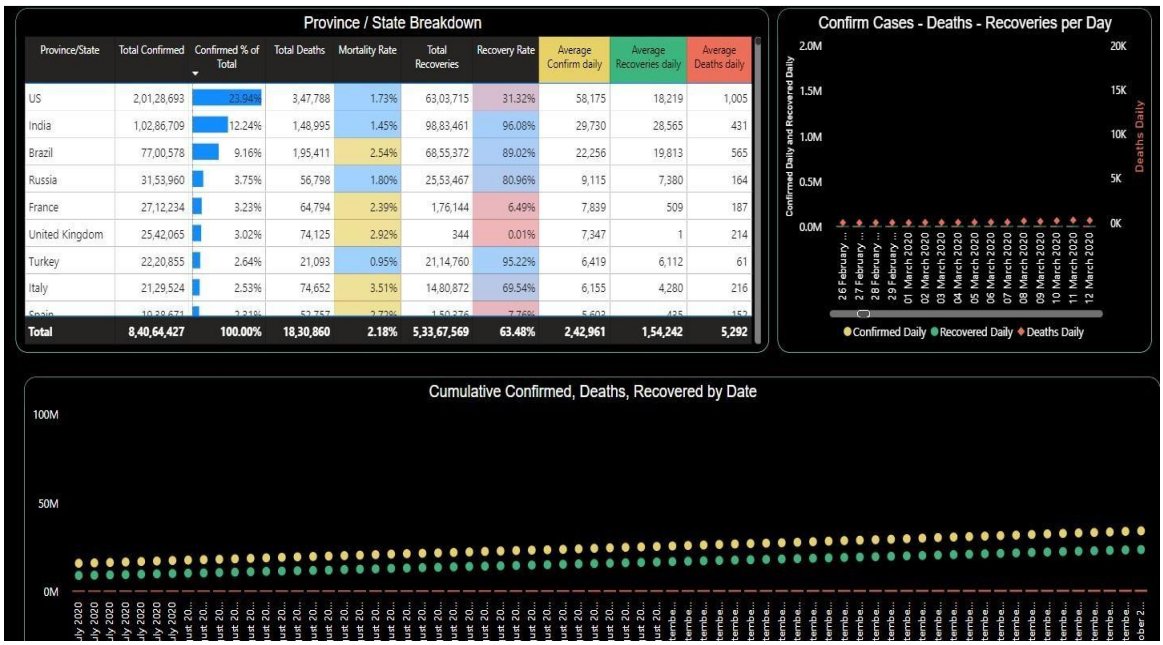



Fig 8.5: Global Data Dashboard (5)

## 8.2. India Data Dashboard

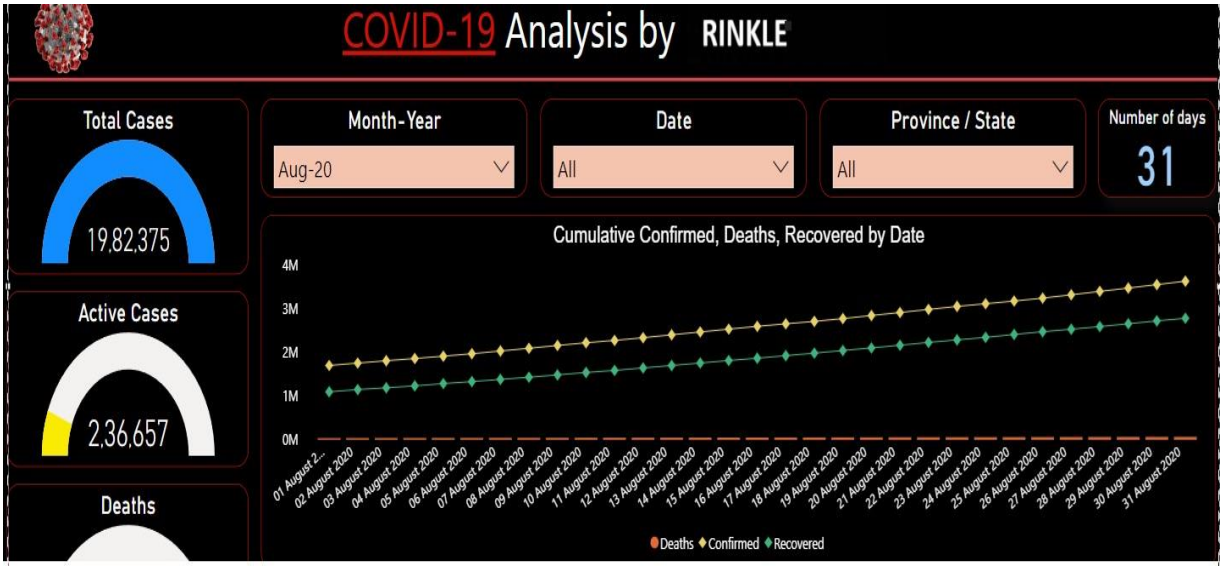



Fig 8.6: Indian Data Dashboard (1)

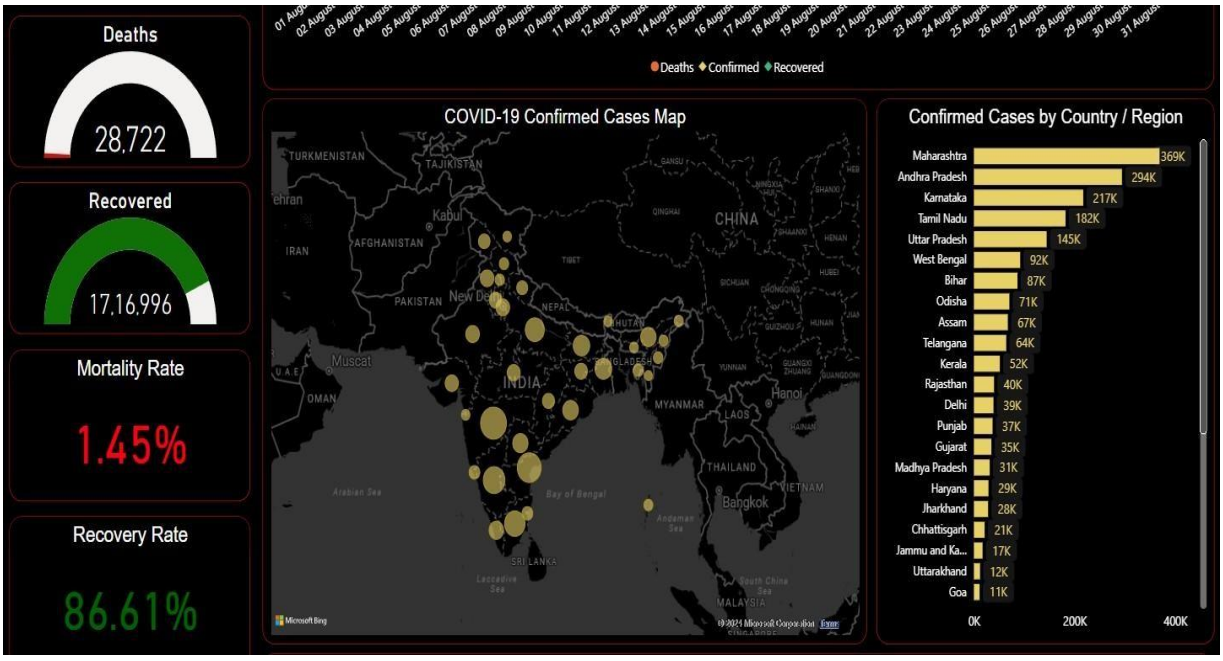
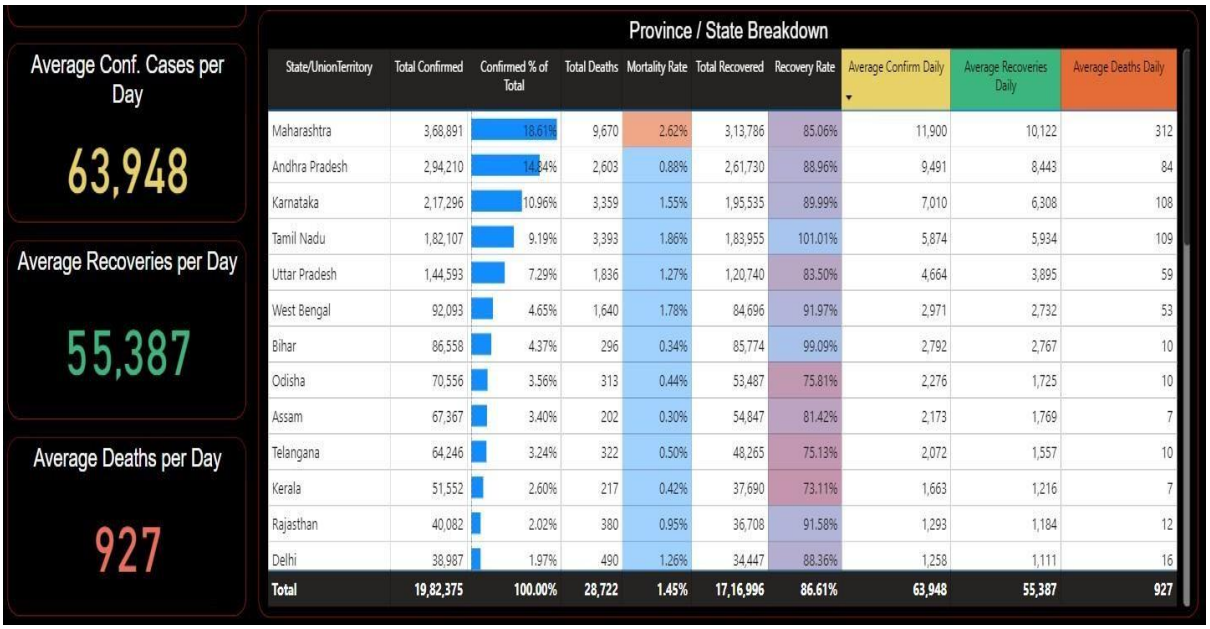



Fig 8.7: Indian Data Dashboard (2)




| State/UnionTerritory | Total Confirmed | Confirmed % of Total | Total Deaths | Mortality Rate | Total Recovered | Recovery Rate | Average Confirm Daily | Average Recoveries Daily | Average Deaths Daily |
|---|---|---|---|---|---|---|---|---|---|
| Maharashtra | 3,68,891 | 18.61% | 9,670 | 2.62% | 3,13,786 | 85.06% | 11,900 | 10,122 | 312 |
| Andhra Pradesh | 2,94,210 | 14.84% | 2,603 | 0.88% | 2,61,730 | 88.96% | 9,491 | 8,443 | 84 |
| Karnataka | 2,17,296 | 10.96% | 3,359 | 1.55% | 1,95,535 | 89.99% | 7,010 | 6,308 | 108 |
| Tamil Nadu | 1,82,107 | 9.19% | 3,393 | 1.86% | 1,83,955 | 101.01% | 5,874 | 5,934 | 109 |
| Uttar Pradesh | 1,44,593 | 7.29% | 1,836 | 1.27% | 1,20,740 | 83.50% | 4,664 | 3,895 | 59 |
| West Bengal | 92,093 | 4.65% | 1,640 | 1.78% | 84,696 | 91.97% | 2,971 | 2,732 | 53 |
| Bihar | 86,558 | 4.37% | 296 | 0.34% | 85,774 | 99.09% | 2,792 | 2,767 | 10 |
| Odisha | 70,556 | 3.56% | 313 | 0.44% | 53,487 | 75.81% | 2,276 | 1,725 | 10 |
| Assam | 67,367 | 3.40% | 202 | 0.30% | 54,847 | 81.42% | 2,173 | 1,769 | 7 |
| Telangana | 64,246 | 3.24% | 322 | 0.50% | 48,265 | 75.13% | 2,072 | 1,557 | 10 |
| Kerala | 51,552 | 2.60% | 217 | 0.42% | 37,690 | 73.11% | 1,663 | 1,216 | 7 |
| Rajasthan | 40,082 | 2.02% | 380 | 0.95% | 36,708 | 91.58% | 1,293 | 1,184 | 12 |
| Delhi | 38,987 | 1.97% | 490 | 1.26% | 34,447 | 88.36% | 1,258 | 1,111 | 16 |
| **Total** | **19,82,375** | **100.00%** | **28,722** | **1.45%** | **17,16,996** | **86.61%** | **63,948** | **55,387** | **927** |

Fig 8.8: Indian Data Dashboard (3)

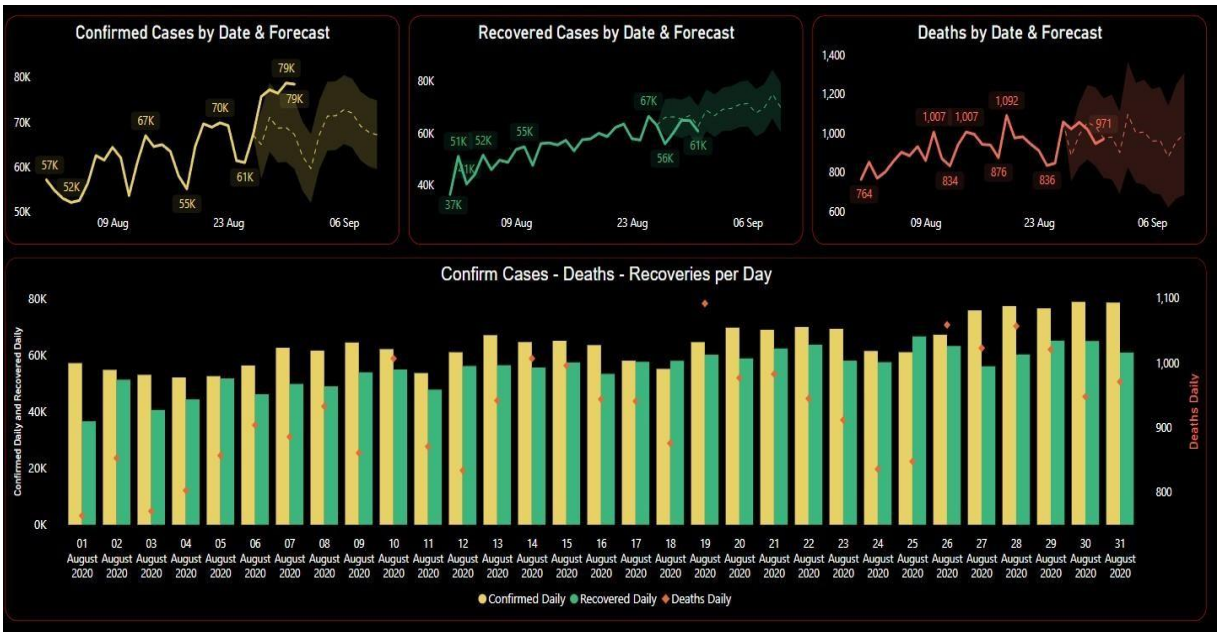



Fig 8.9: Indian Data Dashboard (4)

# CHAPTER 9 Conclusions and Future Scope

## 9.1. Conclusion

Data analytics and visualization tools like Power BI are at the forefront of technological breakthroughs in the 21st century. The spread of COVID-19 (Coronavirus) across the globe has sparked a surge of interest in data analytics and visualizations. People are eager to visualize and comprehend how case counts are escalating, assess the potential impact on themselves and their communities, and understand what actions they can take to mitigate the spread. We use Power BI to create impactful visualizations of common data analyses, aiding us in seeing and understanding our data.

By applying predictive analytics, we aim to enhance decision-making regarding the spread of COVID19 (Coronavirus). Power BI enables users, including scientists and researchers, to quickly access reliable information and even analyse the data themselves. With Power BI, we define discrete and continuous dates, exploring when to use each to elucidate our data. We also delve into mapping, examining how Power BI can utilize various types of geographic data, connect to multiple data sources, and create custom maps.

Power BI is a tool for data visualizations that allows us to connect to almost any database, visualize our data, and gain insights for proper understanding. Using Power BI, we create dashboards, models, interactive visualizations, and thoroughly analyse our data to craft a narrative. Moreover, Power BI provides opportunities for users or key decision-makers to discover patterns in COVID19 data, such as pandemic behaviour, aligning results from machine learning algorithms with Power BI visualizations.

Ultimately, Power BI's primary task is connecting to and extracting stored information from various locations. It can gather information from any platform, including Excel, PDF, Oracle, or Amazon Web

Services databases.

## 9.2. Benefits

The benefits of project are multifaceted and can provide significant insights into the pandemic's impact and progression. The analysis of COVID-19 has brought forth numerous benefits, providing critical insights that have shaped the global response to the pandemic. Through comprehensive data analysis, researchers have been able to track the spread of the virus, understand its impact on different demographics, and evaluate the effectiveness of public health interventions. This has led to more effective strategies for managing public health emergencies, highlighting the importance of a coherent and context-specific approach to such crises. Here are some key benefits:

- **Interactive Visualizations:** Covid-19 Analysis Power BI Project offer interactive dashboards that include charts, maps, and tables, allowing users to explore and analyse data in depth.
- **Trend Analysis:** They enable observation of dynamic trends of confirmed cases, deaths, and recoveries over time, which is crucial for understanding the pandemic's progression and for forecasting future trends.
- **Geographical Distribution:** Projects visualize the geographic distribution of COVID-19 cases, helping to identify highly affected areas and potentially guiding resource allocation.
- **Key Performance Indicators (KPIs):** Monitoring critical KPIs like Case Fatality Rate (CFR), Recovery Rate, and Active Cases helps assess the severity of the pandemic and the effectiveness of response strategies.
- **Data-Driven Decisions:** Risk analysis frameworks guide COVID-19 decisions by assessing risks, managing them, and communicating effectively, which is essential for public health planning and response.
- **Public Health Insights:** Analysis can reveal patterns in the spread of the virus, the effectiveness of preventive measures, and the impact of socio-economic factors on the pandemic's trajectory.
- **Resource Management:** They assist in managing outbreaks with available health resources, ensuring that interventions are timely and effective.

- **Policy Formulation:** By providing a clear picture of the pandemic, the project can inform policymakers, helping them to devise strategies that are coherent, context-specific, and equitable.

In summary, COVID-19 analysis project is invaluable for its ability to turn vast amounts of data into actionable insights, thereby playing a critical role in managing and mitigating the effects of the pandemic.

## 9.3. Future Work

The future work of COVID-19 analysis is poised to evolve significantly as we continue to navigate the pandemic's long-term effects. The focus will shift towards understanding the enduring impacts on public health, the economy, and society at large. Researchers will delve into the long-term health consequences of the virus, including the effects of "long COVID" and the efficacy of various treatment protocols. The future scope of COVID-19 analysis project is quite promising and can be expected to evolve in several key areas.

- **Remote Project Management:** The pandemic has accelerated the shift towards remote work, and this trend is likely to continue in project management. This includes a focus on collaboration tools, clear communication, and goal setting for virtual teams.
- **Advanced Data Analytics:** With the vast amount of data generated by the pandemic, there is a growing need for sophisticated analytics to make sense of it. This could involve the use of machine learning and artificial intelligence to predict future outbreaks and analyse the

effectiveness of interventions.

- **Healthcare System Preparedness:** COVID-19 analysis project can help in preparing healthcare systems for future pandemics by identifying potential weaknesses and areas for improvement in terms of resources, response times, and patient care strategies.

- **Educational Tools:** The data and insights from the project can be used to develop educational materials and programs that raise awareness about infectious diseases and promote public health measures.
- **Global Collaboration:** The pandemic has shown the importance of global cooperation in health crises. Future projects may emphasize collaborative international efforts to share data, research findings, and best practices.
- **Modeling and Simulation:** The use of predictive models to simulate various scenarios and their outcomes will be crucial for planning and decision-making in the face of future health emergencies.
- **Long-term Health Effects:** Project will also likely focus on the long-term health effects of COVID-19, including the study of 'long COVID' and its impact on individuals and healthcare systems.

In essence, COVID-19 analysis project will continue to play a vital role in guiding the world through the pandemic and beyond, shaping the future of public health, policy-making, and global collaboration.

# CHAPTER 10

# Bibliography


- https://api.covid19india.org
- www.wikipedia.com
- www.twitter.com
- https://www.w3schools.com/
- https://stackoverflow.com/
- https://plotly.com/
- https://www.pythonanywhere.com
- https://www.microsoft.com/en-us/power-platform/products/power-bi